

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Ingeniería de Computadores

Identificación de voces e instrumentos en una obra musical

Martín Quevedo Poyal

Dirección

Basilio Sierra Araujo
Izaro Goienetxea Urkizu

Junio de 2026

Resumen

Este Trabajo de Fin de Grado consiste en un acercamiento al problema de separación de fuentes en una obra musical que persigue, dada una pieza musical, obtener mediante diferentes métodos los elementos sonoros de los que se compone.

Con este problema en mente se ha elegido el aprendizaje automático como método para llevar a cabo la tarea. Además de implementar un sistema que permite la separación de una obra musical en sus diferentes componentes sonoros, también se comparan distintos métodos para valorar numéricamente el aprendizaje realizado por el sistema dando así una valoración individual por cada método propuesto y una comparativa que permite visualizar cuál de todos tiene un mejor desempeño sobre la tarea propuesta.

Con esto, y si bien cabe puntualizar que es un trabajo de fin de grado que se ha hecho con los conocimientos que se tienen al terminar el grado y que no pretende competir con aportaciones investigadoras de un nivel más alto, se obtiene finalmente una tabla comparativa de los distintos métodos propuestos, un sistema que permite obtener de dos a cuatro elementos presentes en una obra musical de entrada, y una interfaz gráfica que proporciona accesibilidad a un usuario promedio ajeno al tema. Esto último, además, se ha tratado de llevar a cabo de forma que el usuario pueda entender de forma interactiva lo que está haciendo al variar parámetros o funciones del modelo, añadiendo explicaciones detalladas de cada campo y de los procesos que se están llevando a cabo internamente en el procesado.

ODS

Los Objetivos de Desarrollo Sostenible (*ODS*), establecidos por las Naciones Unidas, constituyen un marco global que busca abordar los grandes desafíos sociales, económicos y medioambientales que enfrenta la humanidad. La investigación y el desarrollo tecnológico, especialmente en campos como la inteligencia artificial y el aprendizaje automático, pueden desempeñar un papel clave en la consecución de estos objetivos. Se definen de acuerdo al diagrama de la Figura 1:



Figura 1: Objetivos de desarrollo sostenible

Este proyecto , centrado en el desarrollo y entrenamiento de modelos de aprendizaje automático con un objetivo concreto, siendo este la identificación y separación de elementos en una obra musical, se alinea con varios de los Objetivos de Desarrollo Sostenible establecidos por las Naciones Unidas. A continuación , se ponen en detalle los objetivos más relevantes con los que este proyecto presenta simpatía y se comenta la aportación de este a cada uno de ellos.

ODS 4: Educación de calidad

Este *ODS* tiene como objetivo garantizar una educación inclusiva, equitativa y de calidad y promover oportunidades de aprendizaje durante toda la

vida para todos.

El sistema presentado en este proyecto, promueve oportunidades de aprendizaje para todos, identificando las funcionalidades más importantes de un modelo de aprendizaje automático , ilustrando las aplicaciones de estos en un ejemplo del mundo real , como puede ser la identificación de elementos en una obra musical y finalmente presentandolas al usuario de forma interactiva y transversal , con explicaciones por parámetro y método que permiten tanto a usuarios experimentados como a primerizos la interacción e introducción a esta tecnología que cada día está mas presente en nuestras vidas.

ODS 3: Salud y bienestar

Este *ODS* busca garantizar una vida sana y promover el bienestar para todos en todas las edades.

Si bien es sabido que el baile (y el disfrute de la música) es una actividad que , como ya se ha demostrado científicamente por multitud de estudios, promueve una mejora de la salud física y emocional del individuo, la separación de fuentes de una obra es ampliamente utilizada para estos propósitos, bien por *DJs* , *Beatmakers* o *Samplers* mediante diferentes técnicas dentro de estos ámbitos. Si bien esto se hacía antiguamente con métodos más rudimentarios , la introducción de la inteligencia artificial y en concreto de las redes neuronales artificiales como herramienta en este tipo de métodos , facilita y mejora el proceso , lo que deriva en el resultado de una mejor herramienta para el artista y por ende en disfrute para el público de este.

Además, este tipo de herramientas son ampliamente utilizadas para la práctica *amateur* de instrumentos musicales , bien para obtener la pista musical de la guitarra en una canción y poder replicarla de oído en casa con tu propio instrumento , o bien para obtener la pista vocal y acompañarla con tu propia partitura.

Estas posibilidades que nos da la herramienta contribuyen lateralmente al bienestar emocional, abriendo bien un abanico de posibilidades creativas en casa o bien un abanico de mejoras en la metodología artística de los “artesanos” de estas disciplinas.

Índice general

Resumen	I
ODS	III
ODS 4: Educación de calidad.....	III
ODS 3: Salud y bienestar	IV
Índice general	V
Índice de figuras	IX
Índice de cuadros	XI
1 Introducción	1
2 Planificación	3
2.1. Alcance	3
2.2. Objetivos	4
2.3. Paquetes de trabajo y tareas	4
2.4. Estimación	8
2.5. Cronograma	9
2.6. Desviaciones respecto a la planificación inicial	9
3 Herramientas utilizadas	13
3.1. Python 3.10.19.....	13
3.2. Librosa	13
3.3. Matplotlib.....	13
3.4. Nussl (<i>Northwestern University Source Separation Library</i>)	14
3.5. Pytorch	14
3.6. Ignite	14
3.7. Streamlit	14
3.8. Torchaudio	14
4 Marco teórico	15
4.1. Sonidos y señales	15

4.1.1.	Naturaleza del sonido	15
4.1.2.	Dominio tiempo frecuencia	18
4.1.2.1.	Magnitud y fase	18
4.1.2.2.	Señales mono y estéreo	19
4.1.2.3.	STFTs (Short-Time Fourier Transform / Trans- formada en Tiempo Corto de Fourier)	20
4.1.2.4.	Espectrogramas	23
4.1.3.	Mezcla de señales y separación de fuentes	25
4.1.3.1.	Mezclas lineales	25
4.1.3.2.	Mezclas coherentes e incoherentes	26
4.1.4.	Representaciones digitales	26
4.1.4.1.	Formatos del audio	26
5	Estado del arte	27
5.1.	Métodos clásicos de separación	28
5.1.1.	Modelado de la señal principal	28
5.1.2.	Modelado del acompañamiento	29
5.1.3.	Acercamientos conjuntos y probabilísticos	29
5.2.	Enfoques basados en aprendizaje profundo	30
5.2.1.	Modelos basados en espectrograma	30
5.2.2.	Modelos <i>waveforms</i> e híbridos	31
5.2.3.	Conjunto de datos y evaluación	31
5.3.	Herramientas y tecnologías actuales	31
6	Desarrollo	33
6.1.	Modelo de separación de fuentes	33
6.1.1.	Base de datos	33
6.1.2.	Datos de entrenamiento	34
6.1.3.	Funciones de pérdida	34
6.1.4.	Arquitectura del modelo	38
6.1.4.1.	Representación de los datos	39
6.1.5.	Entrenamiento	44
6.2.	Interfaz	48
6.2.1.	Diseño de experiencia de persona usuaria	48
6.2.1.1.	Barra lateral de configuración	48
6.2.1.2.	Zona principal de carga	49
6.2.1.3.	Consola y logs.	49
6.2.1.4.	Reproductores.	50
6.2.1.5.	Gráficos.	51
6.2.1.6.	Botones.	52
6.2.1.7.	Modificación de parámetros internos del mode- lo, parámetros de entrenamiento, parámetros de generación de mezclas, de efectos y de la formación de transformadas de Fourier	53

6.2.2.	Casos de uso	56
6.2.2.1.	Parámetros de formación de las transformadas de Fourier	56
6.2.2.2.	Parámetros intrínsecos del modelo	56
6.2.2.3.	Parámetros de efectos para entrenamiento y de audio	57
6.2.2.4.	Parámetros de entrenamiento	57
6.2.2.5.	Parámetros de generación de mezclas	57
6.2.2.6.	Información sobre los parámetros en la interfaz	57
6.2.3.	Visualizaciones en la interfaz	58
6.2.3.1.	<i>Waveforms</i>	58
6.2.3.2.	Espectrogramas	58
6.2.4.	Feedback	59
6.2.4.1.	Logs de consola	59
6.2.4.2.	Mensajes de error	59
6.2.4.3.	Progreso	60
6.2.4.4.	Descarga de fuentes.....	61
7	Integración del sistema	63
7.0.1.	Backend	63
7.0.2.	Comunicación entre módulos	65
8	Evaluación del sistema	67
8.1.	Métricas utilizadas	67
8.2.	Validación del pipeline de evaluación	68
8.3.	Resultados obtenidos	70
8.3.0.1.	Rankings.....	71
8.3.0.2.	Graficos de barras	72
8.3.1.	Discusión y análisis.....	73
8.3.2.	Limitaciones y margen de mejora	77
9	Conclusiones y trabajo futuro	81
9.0.1.	Síntesis del trabajo realizado	81
9.0.2.	Trabajo futuro	81
10	Anexos	83
10.0.1.	Código relevante.....	83
10.0.2.	Configuraciones	84
10.0.3.	Resultados adicionales	85
	Bibliografía	87

Índice de figuras

1.	Objetivos de desarrollo sostenible	III
2.1.	Estructura de desglose de trabajo	5
2.2.	Cronograma inicial del proyecto.	9
2.3.	Cronograma final del proyecto.	10
4.1.	Generación de una onda sonora longitudinal mediante el movimiento de un pistón al final de un tubo [1].	16
4.2.	Representación gráfica de una onda de baja frecuencia y una onda de alta frecuencia.	17
4.3.	Representación gráfica de una onda donde se identifica la amplitud y la amplitud pico a pico.	18
4.4.	Diferencia de fase (desfase) entre dos ondas sinusoidales desplazadas horizontalmente.	19
4.5.	Representación gráfica de las diferencias entre sonido mono y sonido estéreo.	20
4.6.	Proceso de computar una transformada en tiempo corto de Fourier a partir de una onda. Imagen de Bryan Pardo	21
4.7.	Respuesta frecuencial de la ventana Hann.	22
4.8.	Comparativa de efectividad en el problema de separación de fuentes de la ventana Hann en distintos géneros musicales.	22
4.9.	Diferentes generaciones de espectrogramas según el tratamiento de la STFT.	24
4.10.	Espectrograma estándar arriba y espectrograma de Mel abajo.	25
6.1.	Ejemplo de una arquitectura RNN-LSTM.	38
6.2.	Esquema del proceso de inserción de datos al modelo de inferencia de máscaras.	41
6.3.	Media de métricas SDR y SNR por género musical por género en cuatro fuentes.	46
6.4.	Consola y logs de la GUI.	48
6.5.	Zona principal de carga de la GUI.	49
6.6.	Consola y logs de la GUI.	49
6.7.	Reproductores de audio de la interfaz.	50

6.8. Ejemplo de representación gráfica de la fuente vocal.	51
6.9. Botones principales de la interfaz.	52
6.10. Botón de configuración.	53
6.11. Parámetros de formación de transformadas de Fourier modificables en la interfaz visual.	54
6.12. Parámetros de formación de las transformadas de Fourier modifi- cables en la interfaz visual.	55
6.13. Parámetros intrínsecos del modelo modificables en la interfaz visual.	55
6.14. Parámetros modificables de entrenamiento de nuevos modelos en la interfaz visual.	55
6.15. Parámetros modificables de generación de efectos en la interfaz visual.	55
6.16. Parámetros modificables de generación de mezclas en la interfaz visual.	56
6.17. Entrada a la ventana de información sobre los parámetros de la configuración avanzada.	58
6.18. Ejemplo de una forma de onda mostrada en la interfaz al finalizar la separación de una pista.	58
6.19. Ejemplo de un espectrograma mostrado en la interfaz al finalizar la separación de una pista.	59
6.20. Ejemplo de <i>logs</i> mostrados en la consola de la interfaz.	59
6.21. Ejemplo de mensaje de error mostrado en la interfaz al intentar separar una pista sin haber cargado una configuración de modelo previa.	60
6.22. Ejemplo de indicador de espera en la interfaz mientras se está rea- lizando el proceso de separar una pista.	61
6.23. Botón de la interfaz que descarga un archivo wav de fuente en el almacenamiento del usuario.	61
7.1. Diagrama de flujo para el backend.	64
8.1. Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de dos fuentes.	72
8.2. Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de cuatro fuentes.	73

Índice de cuadros

2.1. Enumeración de los objetivos principales	4
2.2. Comparación entre las horas estimadas y las horas reales dedicadas a los bloques y tareas del proyecto.	8
6.1. Funciones de pérdida principales consideradas durante el entrenamiento.	36
6.2. Funciones de pérdida avanzadas y experimentales consideradas durante el desarrollo.....	37
6.3. Configuración arquitectónica de parámetros utilizada en los entrenamientos finales del proyecto.	39
8.1. Tabla para los resultados de separación en 2 fuentes	70
8.2. Tabla para los resultados de separación en 4 fuentes	71
8.3. Ranking para los resultados de separación en 2 fuentes	71
8.4. Ranking para los resultados de separación en 4 fuentes	72

CAPÍTULO Introducción 1



La separación de fuentes musicales o sonoras es, a día de hoy, un campo abierto en el ámbito de la investigación del procesado de señal. Se puede considerar como fuente sonora toda aquella que genere sonido susceptible de ser captado por un sensor de audio, es decir, un micrófono. Por tanto, son fuentes sonoras los instrumentos musicales, la voz, o cualquier fuente de ruido natural o artificial.

El problema en sí mismo se atribuye al ámbito del procesamiento digital de señales y en él se plantea el siguiente reto: desarrollar métodos para que, partiendo de una pieza musical, se obtengan los elementos sonoros que la componen, como por ejemplo obtener a partir de una pieza en formato de archivo digital del género de cantautor donde confluyen una vocal masculina y un instrumento, por ejemplo, una guitarra acústica. El reto consiste en obtener un método que tras su aplicación nos proporcione por un lado una pista con la información sonora de la vocal, y por otro lado otra con la de la guitarra acústica.

Los métodos que se desarrollan para dar solución a este problema tienen aplicaciones en distintos ámbitos relacionados con el procesamiento de audio. Entre ellas se encuentran la creación de versiones instrumentales o de karaoke, la remezcla de obras musicales, la restauración de grabaciones, la identificación y extracción de voces, el análisis automático de música y el apoyo a tareas de transcripción o clasificación sonora. También constituye un problema relevante para el estudio de técnicas de aprendizaje profundo aplicadas a señales temporales.

El trabajo propuesto presenta una dificultad considerable debido a que las fuentes pueden compartir regiones temporales y frecuenciales. Por ejemplo, una voz y un instrumento pueden producir energía en frecuencias similares o sonar simultáneamente y como consecuencia no existe una división directa de la pista que permita recuperar cada fuente sin introducir errores, interferencias o ruido de otros elementos no deseados. Por otro lado, también existen dife-

rentes técnicas de mezcla y masterización que pueden alterar la señal original en función de como interactúan las pistas que componen la canción entre ellas, efectos de reverberación, compresión, ecualización, etc. Estos factores modifican la forma de onda de las diferentes fuentes cuya suma compone la mezcla final y dificultan la generalización, que suele ser el objetivo de los métodos de separación de fuentes.

En el proyecto se ha abordado el problema de la separación de fuentes musicales mediante las herramientas que nos proporcionan los avances tecnológicos de la última década, aplicando inteligencia artificial y aprendizaje automático. La separación de fuentes aún siendo un ámbito muy explorado en el pasado, sigue suponiendo un reto en nuestro tiempo, las pistas pueden compartir regiones temporales y/o frecuenciales, una voz y un instrumento pueden producir energía en frecuencias cercanas del espectro frecuencial y como consecuencia no existir una división directa de ambos que nos permita obtener dos elementos separados. Además, existen factores como algunas técnicas concretas de mezcla o los efectos aplicados de reverberación, compresión, ecualización, etcétera, que hacen que obtener una generalización mediante aprendizaje automático sea una tarea compleja.

El resto de la memoria se aborda de la siguiente manera: en el capítulo 2 se comenta la planificación seguida a lo largo del proyecto y en el capítulo 3 las herramientas que se han utilizado para su desarrollo. Posteriormente, en el capítulo 4, se hace una introducción al marco teórico del procesamiento de señales y del aprendizaje automático, explicando conceptos necesarios para entender el proyecto. En el capítulo 5 se hace un recorrido a lo largo del estado del arte del ámbito. En el capítulo 6 se explican los dos entregables principales que se proporcionan en el trabajo y finalmente en los capítulos ocho y nueve, se dan explicaciones técnicas a cerca de la integración del sistema y de su evaluación respectivamente.

Al final del documento en los capítulos 10 y 11 se encuentran las conclusiones finales y el trabajo futuro, junto con la sección de anexos. A continuación de esta última sección se encuentra la bibliografía.

CAPÍTULO Planificación 2



En este capítulo se presenta la planificación seguida utilizando algunos indicadores y métodos de planificación comunes a los proyectos de ingeniería, entre ellos un alcance inicial del proyecto y unos objetivos a cumplir. Finalmente se comentan las incidencias que han surgido en el desarrollo y que han provocado desvíos de la estimación inicial de tiempo de trabajo para cada fase del proyecto que se había planteado inicialmente.

2.1. Alcance

Este trabajo tiene como alcance la exploración de modelos y distancias en un mismo sistema de aprendizaje automático orientado a la extracción de los elementos sonoros de una pieza musical: voz, bajo, batería y acompañamiento, junto con la implementación de una interfaz gráfica que permite hacer diferentes pruebas.

El trabajo abarca la implementación de distintas funciones de pérdida ($L1$, $L2$, pérdidas en frecuencia, pérdidas de máscara, entre otras), el entrenamiento controlado de modelos con una misma configuración de hiperparámetros, y la evaluación del rendimiento obtenido a través de diferentes métricas.

El proyecto no pretende competir con modelos de última generación en separación musical, sino explorar el impacto de diferentes estrategias de entrenamiento y funciones de pérdida en arquitecturas más simples, trabajando con *datasets* ya creados y recursos de cómputo reducidos.

2.2. Objetivos

El principal objetivo del proyecto es desarrollar una herramienta capaz de separar los diferentes elementos presentes en una pista de audio.

1. Objetivos principales:

Desarrollo de un sistema de aprendizaje automático capaz de generar una salida que contenga los elementos musicales presentes en una pista de audio de entrada. El desarrollo planteado para este objetivo se explica en la Tabla 2.1

Tabla 2.1: Enumeración de los objetivos principales

Objetivo	Definición	Condición para considerarlo completado
Primera implementación sencilla		
Separación de dos fuentes	Implementar un sistema que proporcione resultados satisfactorios para el problema planteado, tomando como entrada una canción compuesta por dos elementos.	Obtener dos archivos de audio de salida sin ruido ni interferencias, que presenten una correspondencia directa con los dos elementos presentes en la pista de entrada.
Implementación final más compleja		
Separación de cuatro fuentes	Escalar el sistema previamente implementado para obtener resultados satisfactorios cuando la entrada sea una canción compuesta por cuatro o más elementos.	Obtener cuatro archivos de audio de salida sin ruido ni interferencias, correspondientes a las agrupaciones de bajo, batería, voz y otros. La categoría “otros” incluye los elementos que no pertenecen a las categorías de bajo, batería o voz.

2. Objetivos secundarios:

Aunque no formaba parte de los objetivos iniciales y se plantea como una propuesta adicional, condicionada al cumplimiento de la planificación.

Se considera también el desarrollo de una interfaz gráfica sencilla que permita cargar audios de prueba, obtener sus *stems* (término inglés hace referencia a las subpistas o fuentes que componen una mezcla musical) separados y almacenar los resultados en forma de archivos de audio y representaciones gráficas con un enfoque didáctico.

2.3. Paquetes de trabajo y tareas

La organización por tareas del proyecto ha sido desglosada en una Estructura de Desglose de Trabajo o EDT que se muestra en la Figura 2.1.

2.3. Paquetes de trabajo y tareas

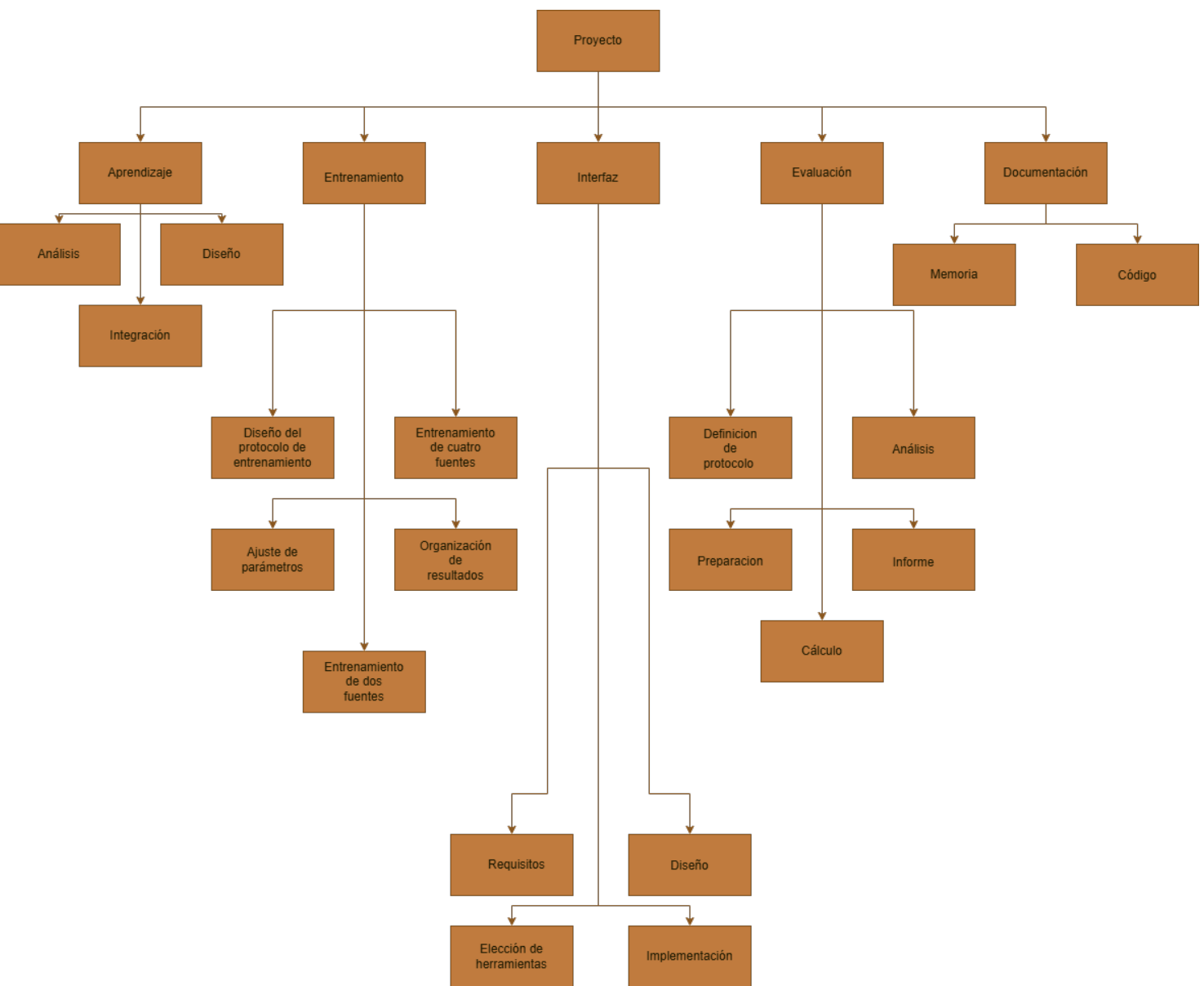


Figura 2.1: Estructura de desglose de trabajo

Esta estructura está dividida en distintos paquetes de trabajo de la siguiente manera:

- **PT 1. Aprendizaje y desarrollo inicial.**
 - **T 1.1. Análisis del problema e investigación técnica.**

Estudio del problema de separación de fuentes, revisión de conceptos de audio digital y análisis de las herramientas necesarias para el desarrollo del sistema.

- **T 1.2. Implementación inicial en código de un modelo de separación de fuentes y de las funciones de pérdida.**

Identificación de limitaciones y necesidades del proyecto, Implementación del modelo elegido de acuerdo a estas e incorporación de las funciones de pérdida propuestas.

- **T 1.3. Diseño del preprocesamiento.**

Definición del proceso de preparación de los datos y transformación de señales.

■ PT 2. Entrenamiento.

- **T 2.1. Diseño del protocolo de entrenamiento.**

Agrupación de muestras musicales elegidas en un conjunto de datos para entrenamiento y definición de estrategia para este , configuraciones evaluadas y criterios de comparación.

- **T 2.2. Ajuste de los parámetros de entrenamiento.**

Estudio y elección de hiperparámetros. Integración de una configuración de estos para que se cumplan los requisitos de las dos variantes posibles del modelo planteadas a entrenar en las dos siguientes fases.

- **T 2.3. Entrenamiento de dos fuentes.**

Entrenamiento de modelos orientados a separar una mezcla en voz y acompañamiento.

- **T 2.4. Entrenamiento de cuatro fuentes**

Entrenamiento de modelos orientados a separar una mezcla en batería, voz, bajo y resto.

- **T 2.5. Organización de los resultados.**

Recopilación, ordenación y preparación de los resultados generados durante los entrenamientos.

■ PT 3. Implementación de la interfaz.

- **T 3.1. Análisis de requisitos.**

Definir funcionalidades que debe ofrecer la interfaz.

- **T 3.2. Elección de herramientas.**

Seleccionar las tecnologías a utilizar en la implementación teniendo en cuenta la facilidad de uso y la integración y compatibilidad con el sistema desarrollado.

- **T 3.3. Diseño del flujo.**

Definir la secuencia de interacción del usuario con la aplicación.

- **T 3.4. Implementación.**

Desarrollo de la interfaz, conexión con el modelo y validación de funcionalidades.

- **PT 4.Evaluación.**

- **T 4.1. Definición de protocolo de evaluación.**

Establecer cómo se van a evaluar los modelos, datos a utilizar y métricas a calcular.

- **T 4.2. Preparación de conjunto de evaluación**

Seleccionar y organizar los audios a utilizar para medir el rendimiento final.

- **T 4.3. Cálculo de métricas.**

Comprende el cálculo de métricas mediante scripts para cada modelo.

- **T 4.4. Análisis y representación de resultados.**

Interpretación de resultados obtenidos, comparación entre modelos y elaboración de gráficas.

- **T 4.5. Elaboración de conclusiones y de trabajo futuro.**

Recoger conclusión final en vista de los resultados y las limitaciones que estos presentan.

- **PT 5. Documentación**

- **T 5.1. Redacción de la memoria.**

Comprende escritura de la memoria y comunicación con los tutores para la corrección de esta.

- **T 5.2. Documentación del código.**

Revisión y limpieza del repositorio de código, organización de archivos y documentación básica.

2.4. Estimación

Inicialmente la planificación del proyecto plantea cinco partes divididas cada una en tareas con su pertinente estimación temporal como se muestra en la Tabla 2.2.

Tabla 2.2: Comparación entre las horas estimadas y las horas reales dedicadas a los bloques y tareas del proyecto.

Trabajo Fin de Grado	Horas estimadas	Horas reales
PT 1 Aprendizaje y desarrollo inicial	62	76
T 1.1 Análisis del problema e investigación técnica	24	22
T 1.2 Integración del modelo y de las funciones de pérdida	22	40
T 1.3 Diseño del preprocesamiento	16	14
PT 2 Entrenamiento	150	137
T 2.1 Diseño del protocolo de entrenamiento	10	10
T 2.2 Ajuste de los parámetros de entrenamiento	28	36
T 2.3 Entrenamiento de dos fuentes y análisis de resultados	22	40
T 2.4 Entrenamiento de cuatro fuentes y análisis de resultados	30	50
T 2.5 Organización de los resultados	10	8
PT 3 Implementación de la interfaz	44	39
T 3.1 Análisis de requisitos	8	6
T 3.2 Elección de herramientas	6	5
T 3.3 Diseño del flujo	14	12
T 3.4 Implementación	16	16
PT 4 Evaluación final	80	150
T 4.1 Definición del protocolo de evaluación	10	11
T 4.2 Preparación del conjunto de evaluación	12	12
T 4.3 Cálculo de métricas	4	4
T 4.4 Análisis y representación de los resultados	14	14
T 4.5 Elaboración de conclusiones y trabajo futuro	12	46
PT 5 Documentación	80	150
T 5.1 Redacción y revisión de la memoria final	18	58
T 5.2 Limpieza y documentación del código	4	6
TIEMPO TOTAL	300	400

La estimación inicial de tiempo dedicado al proyecto asciende a aproximadamente 350 horas de trabajo efectivo. Esta cifra incluye las tareas de análisis, implementación, depuración, supervisión de entrenamientos, evaluación y documentación. No se contabiliza íntegramente el tiempo durante el cual los modelos se entrenan de forma automática, sino únicamente el esfuerzo dedicado a preparar, supervisar y analizar dichas ejecuciones.

2.5. Cronograma

En la Figura 2.2 se puede ver el cronograma inicial.

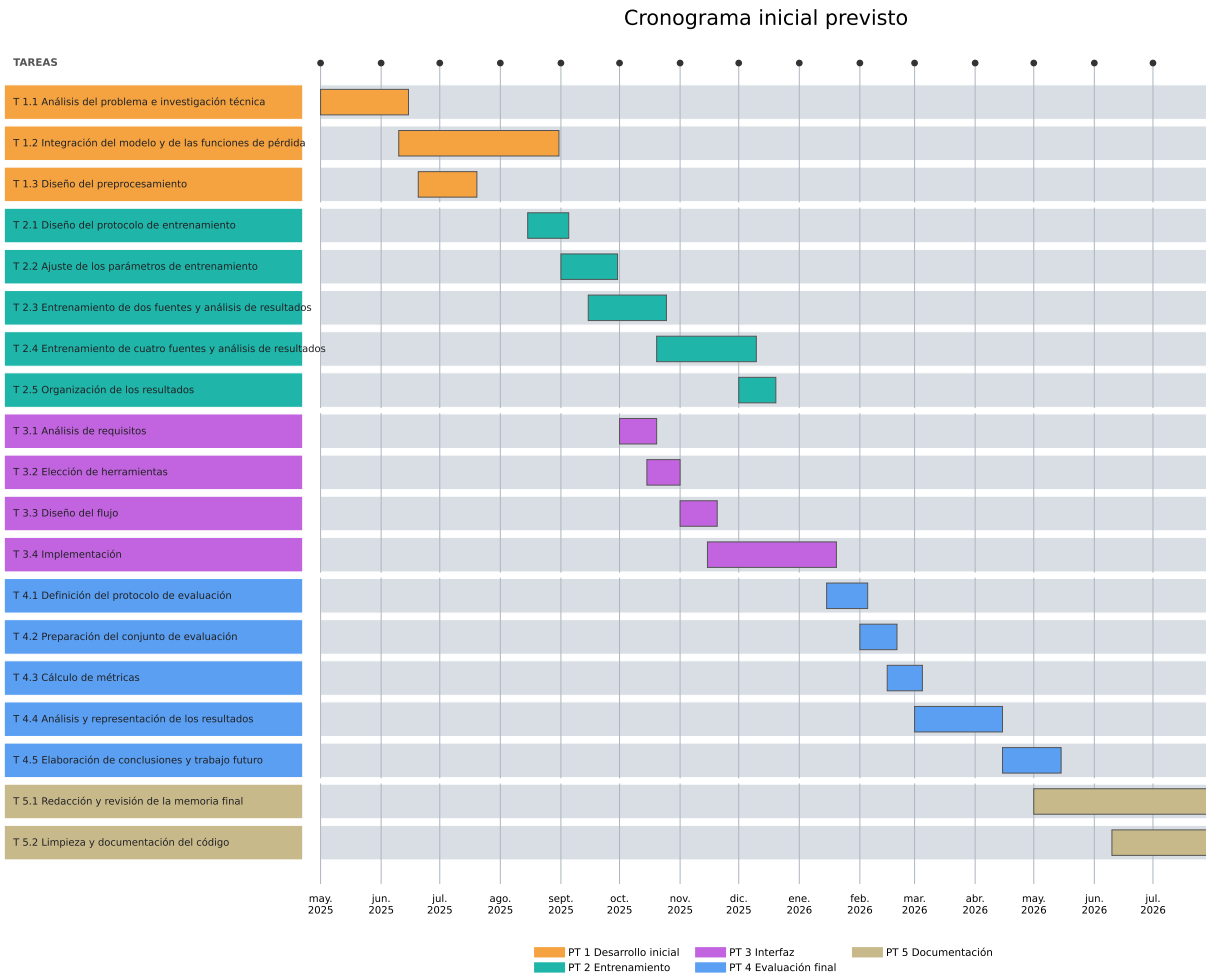


Figura 2.2: Cronograma inicial del proyecto.

2.6. Desviaciones respecto a la planificación inicial

En la planificación se asumía que, una vez entrenados los modelos, la evaluación consistiría principalmente en ejecutar las métricas definidas, probar los modelos generados y valorar los resultados obtenidos. Sin embargo, durante el desarrollo del proyecto fueron surgiendo varios problemas que modificaron el desarrollo previsto. Finalmente la ejecución del proyecto se puede ver en el cronograma de la Figura 2.3, las incidencias que han causado los retrasos en la planificación son las enumeradas a continuación.

2. PLANIFICACIÓN

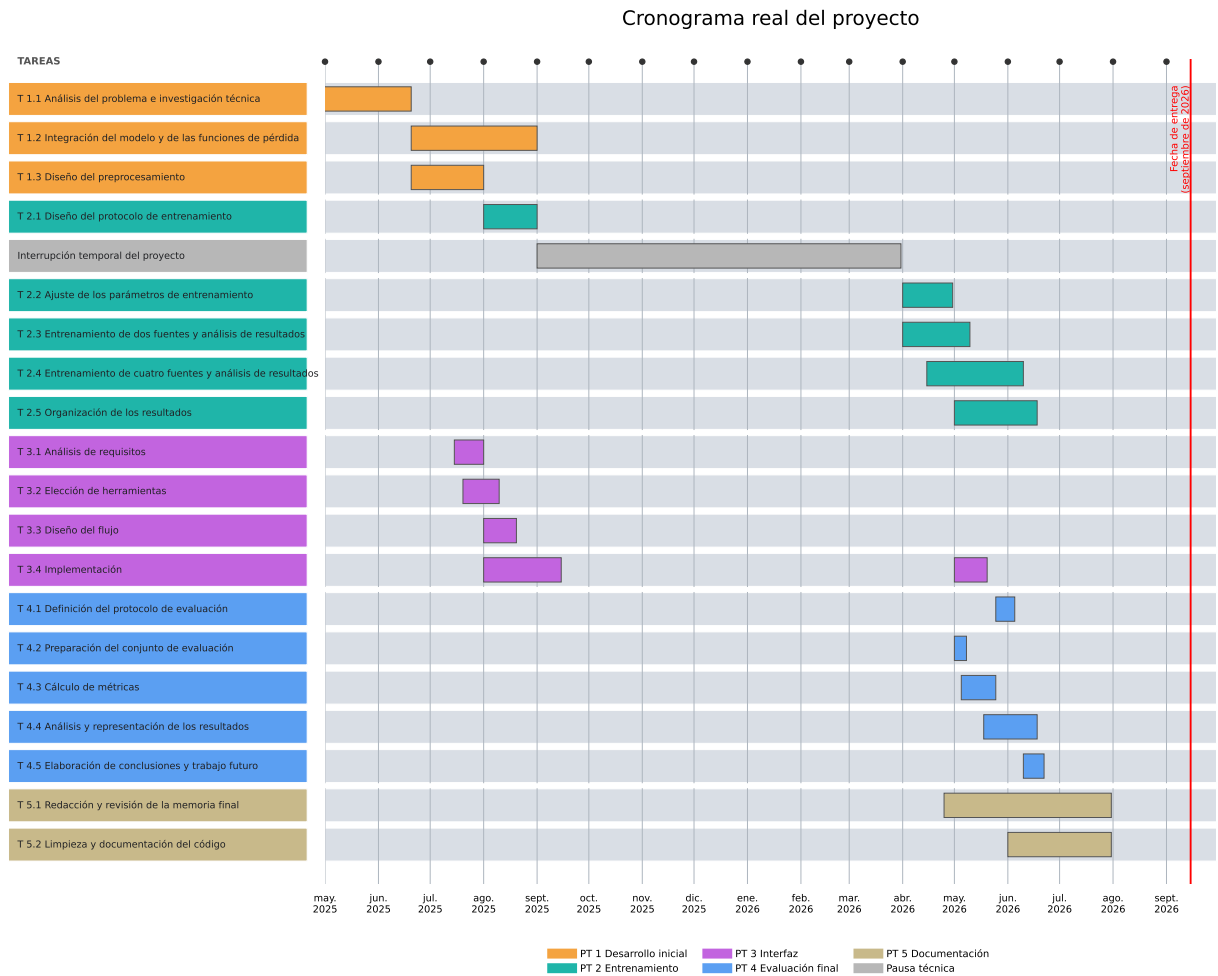


Figura 2.3: Cronograma final del proyecto.

Las incidencias que causaron este aumento en el tiempo dedicado se pueden resumir en los siguientes tres grupos:

- **Incidencias por complejidad de la implementación:**

Errores en la implementación de carga de puntos de guardado y la implementación en el entrenamiento, y dimensiones en los valores calculados causaron gran impacto en las tareas 1.2, 1.3 y 2.1.

- **Incidencias en el entrenamiento:**

Errores de parámetros, carga de datos y de modelos y de despliegue se terminaron por realizar tres tandas diferentes de entrenamientos, lo que causó gran impacto en las tareas 2.1, 2.2, 2.3 y 2.4.

- **Incidencias por coste computacional:**

2.6. Desviaciones respecto a la planificación inicial

Entrenar todas las funciones de pérdida resultó bastante más costoso de lo previsto inicialmente y causó gran impacto en las tareas 2.3 y 2.4.

CAPÍTULO 3

Herramientas utilizadas



En esta sección se explicarán las herramientas utilizadas en el proyecto.

3.1. Python 3.10.19

Python es un lenguaje de alto nivel de programación interpretado cuya filosofía hace hincapié en la legibilidad de su código. Se trata de un lenguaje de programación multiparadigma, ya que soporta parcialmente la orientación a objetos, programación imperativa y, en menor medida, programación funcional. Es un lenguaje interpretado, dinámico, multiplataforma.. Sus ventajas principales incluyen su sintaxis sencilla y legible, su enorme ecosistema de librerías para Inteligencia Artificial y ciencia de datos.

3.2. Librosa

Librería de *Python* más popular para análisis de audio. Construida sobre *NumPy*, *SciPy* y *Matplotlib*. Utilizada para dar soporte a las transformaciones en dominio tiempo-frecuencia, visualización de ondas y espectrogramas, y cargas de audios.

3.3. Matplotlib

Librería estándar de visualización en *Python*. Proporciona principalmente herramientas para creación de gráficos. En el proyecto se ha utilizado para graficar señales, espectrogramas, valores de métricas y resultados.

3.4. Nussl (*Northwestern University Source Separation Library*)

Librería de Python especializada en separación de fuentes de audio, proporciona métodos de preprocesamiento de audio, implementación y comparación de algoritmos de separación de fuentes, modelos pre-entrenados, métricas ... En el proyecto se ha utilizado para cargas de audio *wav* y *mp3*, obtención y reconstrucción de espectrogramas, construcción del modelo, inferencia de las fuentes, generación de STFTs y obtención de métricas.

3.5. Pytorch

Librería de aprendizaje profundo en *Python* desarrollada por Meta (previamente *Facebook*) enfocada en la creación, entrenamiento y despliegue de redes neuronales. Se utiliza en el trabajo para cargar los modelos entrenados, trabajar con los tensores y con las funciones de pérdida, proporcionar métodos de entrenamiento y *backpropagation*..

Pytorch es básicamente la herramienta sobre la que se apoya la ejecución del proyecto.

3.6. Ignite

Librería de alto nivel construida sobre *PyTorch* que facilita el entrenamiento de modelos mediante abstracciones que proporcionan métodos ya implementados para entrenamiento, validación, *checkpoints* ... Además de para generación de métricas, en el proyecto se utiliza principalmente para manejar bucles de entrenamiento/validación, guardado/carga de *checkpoints*, *early-stopping*, registro de métricas y *logs*.

3.7. Streamlit

Librería de *Python* pensada para creación de interfaces gráficas y aplicaciones *web* interactivas de manera simple y enfocada a ciencia de datos e IA. En el proyecto proporciona la implementación de lo que sería la parte del *Frontend* del proyecto.

3.8. Torchaudio

Librería oficial de *Python* centrada en el procesamiento de audio y voz proporcionando *datasets*, transformaciones, y modelos pre-entrenados. Se utiliza en el proyecto para carga de audios, transformaciones, *Data Augmentation*, acceso al dataset ...

CAPÍTULO

Marco teórico

4



En esta sección se van a introducir una serie de conceptos teóricos considerados relevantes para el desarrollo del proyecto. El capítulo se divide en dos grandes bloques, el primero dedicado a conceptos importantes en el campo del sonido y de las señales acústicas, y el segundo en relación al aprendizaje automático.

4.1. Sonidos y señales

4.1.1. Naturaleza del sonido

El sonido es una perturbación mecánica que se propaga en medios elásticos (aire, agua o sólidos) y que percibimos a través del sistema auditivo.

Algunos conceptos importantes:

■ 1. Ondas Sonoras

Las ondas sonoras son oscilaciones longitudinales que provocan variaciones en la presión del medio, zonas más densas (compresiones) y zonas menos densas (rarefacciones). Se generan por una fuente vibratoria, como una cuerda, un altavoz, cuerdas bucales o una gota de agua y esa vibración se transmite por colisión entre partículas del medio [1] como por ejemplo, un pistón que al moverse al final de un tubo genera ruido en la Figura 4.1.

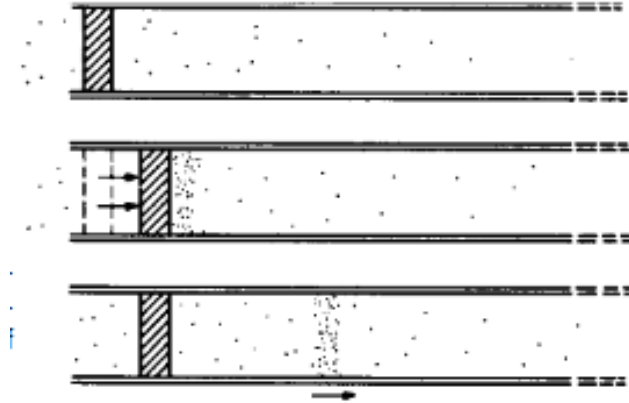


Figura 4.1: Generación de una onda sonora longitudinal mediante el movimiento de un pistón al final de un tubo [1].

■ 2. Frecuencia y tono

La frecuencia es el número de ciclos completos de la onda por segundo, en la Figura 4.2 se puede ver gráficamente la diferencia entre dos ondas con una frecuencia diferente. Se mide en hercios (Hz) y determina el tono de lo que escuchamos. Las notas tienen sus propias frecuencias, por ejemplo $440Hz$ corresponde a La_4 , la voz humana suele ir en el rango 128-256 Hz , los ultrasonidos se consideran los superiores a 20 kHz y los infrasonidos los inferiores a 20 Hz [2]. El rango de audición humana oscila entre los 20 Hz y los 20 kHz .

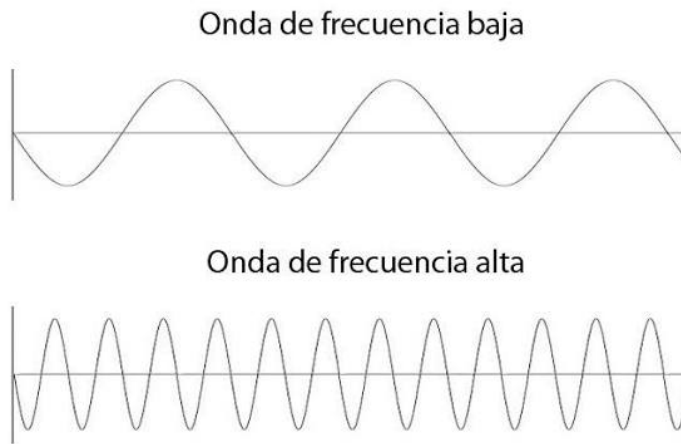


Figura 4.2: Representación gráfica de una onda de baja frecuencia y una onda de alta frecuencia.

Por otro lado, el tono está directamente relacionado con el término de *frecuencia fundamental*, la frecuencia más baja de todas las frecuencias de un sonido que indica la nota que hemos escuchado, aunque en los sonidos normalmente intervienen armónicos, que son múltiplos de la frecuencia fundamental lo que afecta también al timbre, que es una característica del sonido que nos permite diferenciar una misma nota tocada en una guitarra, en un violín o en una flauta.

■ 3. Amplitud e intensidad

La amplitud corresponde al máximo desplazamiento de las partículas en el medio respecto a su posición de equilibrio, reflejándose en el sonido como la magnitud de variación de presión o vibración. Se percibe como *volumen o sonoridad*, ondas con mayor amplitud suenan más fuertes. Se muestra la representación gráfica de una onda donde se identifica la amplitud y la amplitud pico a pico en la Figura 4.3

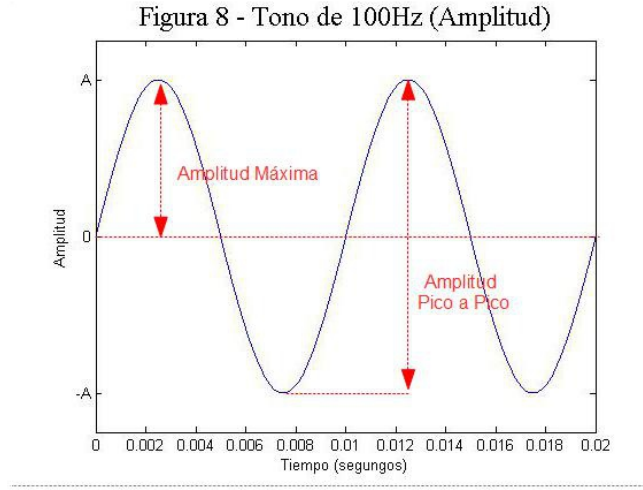


Figura 4.3: Representación gráfica de una onda donde se identifica la amplitud y la amplitud pico a pico.

La intensidad sonora mide la potencia transmitida por unidad de área (W/m^2) en la dirección del sonido y está relacionada con la amplitud al cuadrado[3]:

$$I \propto A^2$$

4.1.2. Dominio tiempo frecuencia

En esta sección se explican conceptos importantes para llevar a cabo el proyecto y relacionados con el dominio tiempo-frecuencia, como magnitud y fase, señales mono y estéreo, transformadas de Fourier y espectrogramas.

4.1.2.1. Magnitud y fase

La magnitud y fase son los dos principales componentes sonoros en los que descomponemos una señal. Se obtienen mediante la aplicación de la *transformada en tiempo corto de Fourier*. La magnitud es el módulo del coeficiente complejo de la STFT (*Short Time Fourier Transform*, siglas para Transformada de Tiempo corto de Fourier en inglés). Se explica dos subsecciones más abajo ya que es una explicación que requiere de la explicación de la magnitud y fase.

La expresión matemática que representa el módulo o valor absoluto del coeficiente complejo en el dominio tiempo-frecuencia es la siguiente:

$$|X(t, f)|$$

Mientras que la fase determina el desfase temporal relativo de esa componente, dependiente del tiempo t y de la frecuencia f :

$$\angle X(t, f)$$

En la Figura 4.4 se puede ver un ejemplo de la diferencia de fase (desfase) entre dos ondas sinusoidales desplazadas horizontalmente.

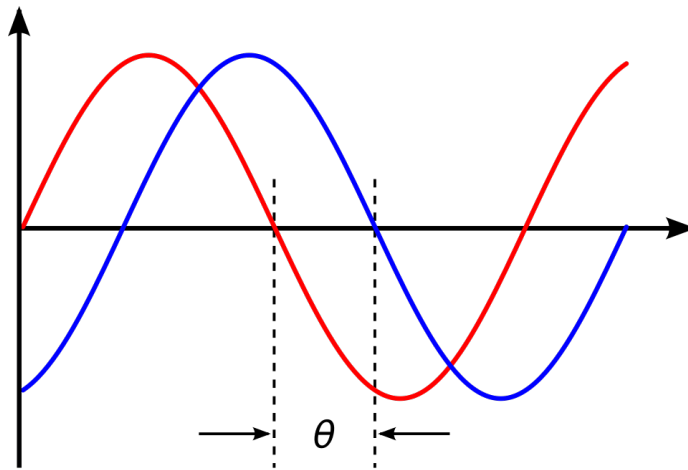


Figura 4.4: Diferencia de fase (desfase) entre dos ondas sinusoidales desplazadas horizontalmente.

Aunque la magnitud es frecuentemente usada como entrada a modelos de separación, la fase es esencial para reconstruir una señal con facilidad, interesante en el contexto del problema que se aborda en este proyecto.

4.1.2.2. Señales mono y estéreo

Las señales mono se presentan en un solo canal, mientras que las estéreo presentan dos canales, izquierdo y derecho, por lo que crean una sensación espacial y de amplitud dinámica como se muestra en la Figura 4.5.

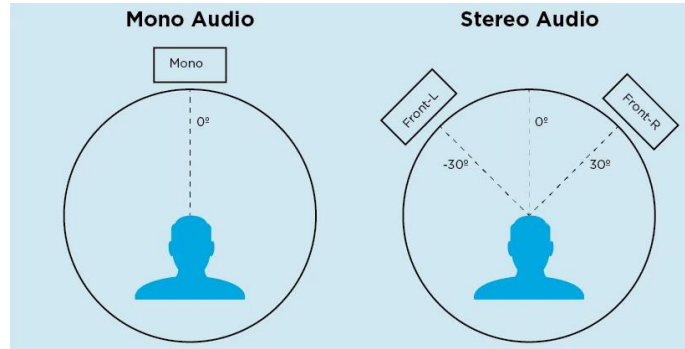


Figura 4.5: Representación gráfica de las diferencias entre sonido mono y sonido estéreo.

4.1.2.3. STFTs (Short-Time Fourier Transform / Transformada en Tiempo Corto de Fourier)

La Transformada en Tiempo corto de Fourier es un método que captura mediante ventanas sucesivas de diferentes formas secciones de una onda y que permite analizar la evolución de su frecuencia a lo largo del tiempo. Se define de acuerdo a la siguiente fórmula:

$$X(n, \omega) = \sum_{m=-\infty}^{\infty} x[m] \cdot w[m - n] \cdot e^{-j\omega m}$$

Donde:

- $x[m]$ es la señal de audio original en tiempo discreto. Cada valor representa una muestra digital de la señal.
- m es el índice de muestra utilizado para recorrer la señal.
- $w[m - n]$ es la función de ventana desplazada hasta la posición temporal n . Esta ventana selecciona el fragmento de la señal que se analiza en cada instante.
- n representa la posición temporal de la ventana dentro de la señal.
- $e^{-j\omega m}$ es una exponencial compleja utilizada para analizar la presencia de una determinada frecuencia en el fragmento seleccionado.
- j es la unidad imaginaria, definida mediante $j^2 = -1$.
- ω es la frecuencia angular digital, normalmente expresada en radianes por muestra.

El método para aplicar la transformación discreta de Fourier consiste en la captura mediante ventanas sucesivas, no necesariamente triangulares, pero sí normalmente, en movimiento L de secciones de la onda trasladada a lo largo de la onda (L , $2L$, $3L$...). Finalmente, se aplica la transformada discreta de Fourier a cada segmento, obteniendo una representación de la evolución temporal del contenido frecuencial de la señal, como se muestra en la Figura 4.6

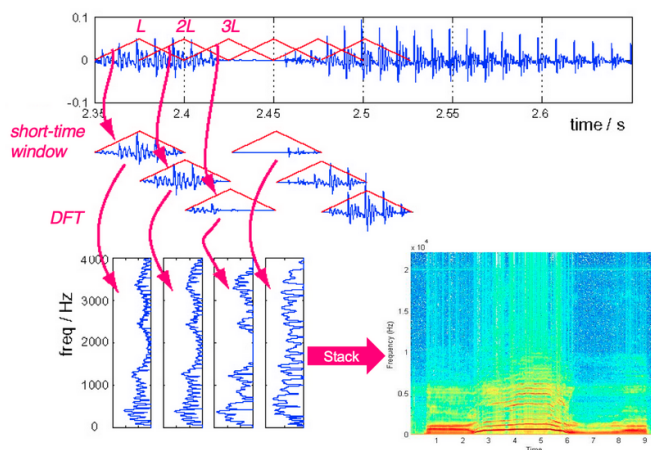


Figura 4.6: Proceso de computar una transformada en tiempo corto de Fourier a partir de una onda. Imagen de Bryan Pardo

Cada celda resultante de la transformada es compleja (contiene magnitud y fase) y la transformada en general es invertible, por lo que se puede obtener la onda inicial a partir de la representación.

Los parámetros importantes en la transformada de Fourier son:

- 1. Forma de la ventana: gaussiana, rectangular, triangular, cuadrática ... Determina la cantidad de información que se captura en una ventana,
- 2. Longitud de la ventana: determina la precisión con la que se verá cuándo ocurren los eventos o cuáles son exactamente las frecuencias
- 3. Distancia entre ventanas: salto entre ventanas consecutivas, tiene relación directa con el equilibrio entre resolución temporal, precisión y costo computacional.

En el proyecto se computa la transformada con la aplicación de una ventana Han que se caracteriza por un lóbulo principal ancho y lóbulos laterales pequeños que decaen rápidamente. Esto se traduce en extremos de la señal suavizados a cero, lo que elimina las discontinuidades en los bordes antes de aplicar la Transformada Rápida de Fourier. El lóbulo principal es el doble de

ancho que el de una ventana rectangular tradicional, lo que disminuye ligeramente la resolución espectral, y los laterales atenúan -32dB y 18dB por octava, respectivamente. En la Figura 4.7 se puede ver la graficación de una ventana Hann y su respuesta frecuencial.

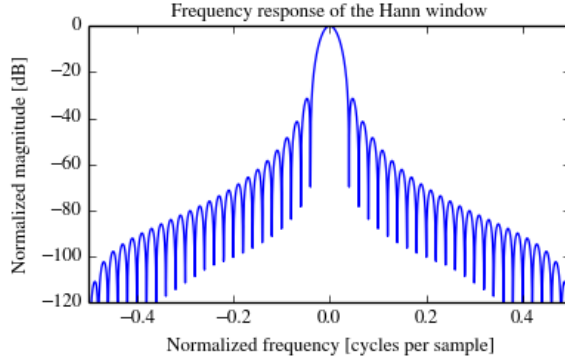


Figura 4.7: Respuesta frecuencial de la ventana Hann

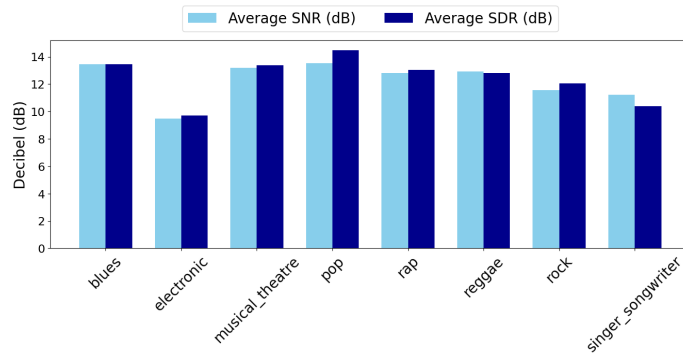


Figura 4.8: Comparativa de efectividad en el problema de separación de fuentes de la ventana Hann en distintos géneros musicales.

4.1.2.3.1. iSTFT Otro término a tener en cuenta en relación con la Transformada en Tiempo corto de Fourier es su versión inversa, la *iSTFT*, donde se reconstruye una señal de audio u onda a partir de su representación en el dominio tiempo-frecuencia

4.1.2.4. Espectrogramas

Es la representación visual de la magnitud de la STFT, y contrapone gráficamente tiempo y frecuencia haciendo un gradiente de color a más oscuro según se incrementa la más intensidad/magnitud presente la señal en ese punto.

Existen diferentes tipos, en función de cómo se traten los respectivos elementos de la STFT (magnitud, cuadrado, logarítmico, logarítmico cuadrado). Se puede observar cómo varía su generación en función del tratamiento de la STFT en la Figura 4.9 y que se diferencian en lo siguiente:

- El espectrograma de magnitud muestra directamente la amplitud de cada frecuencia. Se diferencia del espectrograma de potencia en que la diferencia entre una componente débil de la señal y una fuerte es menor y de los dos gráficos inferiores porque al utilizar una escala lineal las componentes más fuertes dominan el color y las más débiles quedan oscuras.
- El espectrograma de potencia representa el cuadrado de la magnitud. Se diferencia del gráfico de magnitud porque en este los valores mayores que uno aumentan considerablemente y los pequeños se reducen todavía más, es decir, existe un rango mas amplio entre valores fuertes y débiles.
- El espectrograma logarítmico aplica un logaritmo a la magnitud. Se diferencia de los espectrogramas de magnitud superiores en que al aplicarse un logaritmo se reduce el rango dinámico y con ello la diferencia visual entre componentes fuertes y débiles y por eso se muestra con más brillo y texturas.
- El espectrograma logarítmico potencia aplica un logaritmo a la potencia de la magnitud. Se diferencia de los espectrogramas superiores en que se recuperan los valores que quedan ocultos en la escala lineal, y del logaritmo de magnitud en que al apartir de los valores elecados al cuadrado las regiones energéticas resaltan más y las débiles quedan mas ocultas.

4. MARCO TEÓRICO

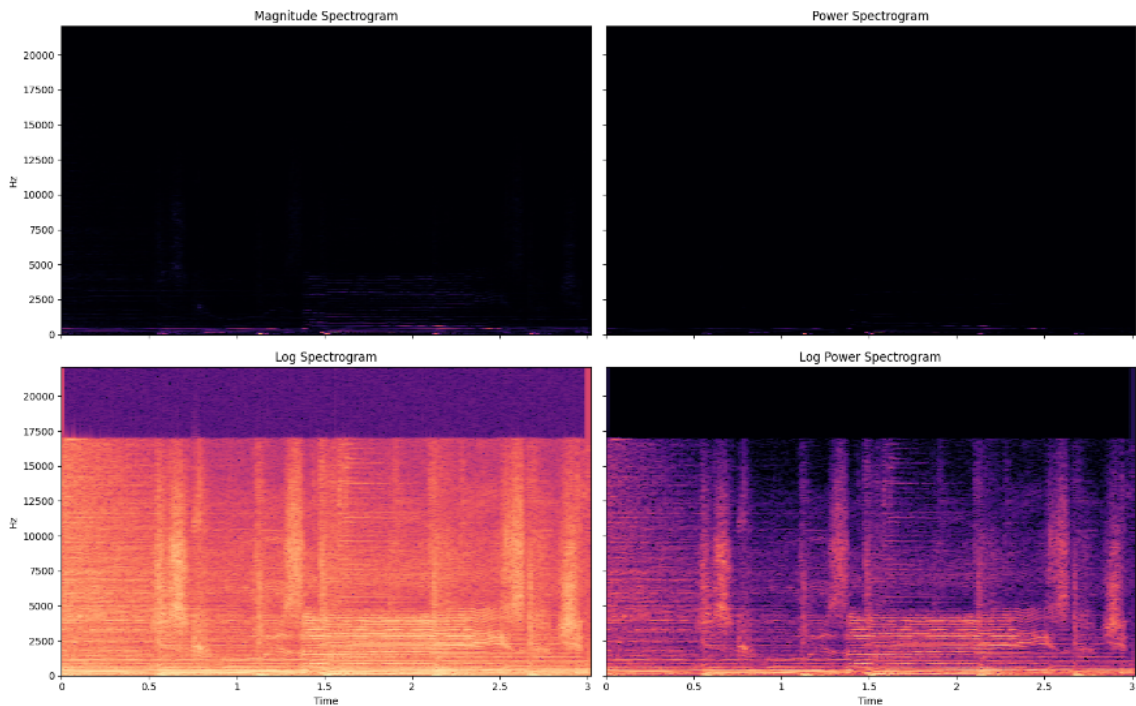


Figura 4.9: Diferentes generaciones de espectrogramas según el tratamiento de la STFT.

Por lo general en procesamiento de audio se utiliza un tipo especial: los espectrogramas de Mel. Un espectrograma de Mel es similar a un espectrograma común en el sentido de que muestra el contenido de frecuencia de una señal de audio a lo largo del tiempo pero en un eje de frecuencia diferente. En un espectrograma estándar, el eje de frecuencia es lineal y se mide en hercios (Hz), pero sin embargo el sistema auditivo humano es más sensible a los cambios en las frecuencias bajas que en las frecuencias altas, y esta sensibilidad disminuye de manera logarítmica a medida que la frecuencia aumenta. La escala mel es una escala perceptual que aproxima la respuesta de frecuencia no lineal del oído humano. [4]. En la Figura 4.6 se muestra la diferencia en la representación visual de ambos métodos.

Si bien el espectrograma común que se utiliza en procesamiento de audio es el de Mel, en este trabajo se ha optado por mostrar el contenido de la frecuencia de una señal de audio utilizando espectrogramas estándar. La principal razón detrás de esta elección es porque el espectrograma Mel representa sonido de una manera más afín a la percepción humana, de manera que realmente se pierde información, mientras que el espectrograma STFT lineal conserva una correspondencia exacta entre frecuencia máscara magnitud y fase. Como el sistema desarrollado trabaja estimando máscaras a aplicar sobre la magnitud STFT de la mezcla, la representación lineal es la opción natural: la máscara se

estima en el mismo espacio donde se aplica y donde posteriormente se realiza la reconstrucción.

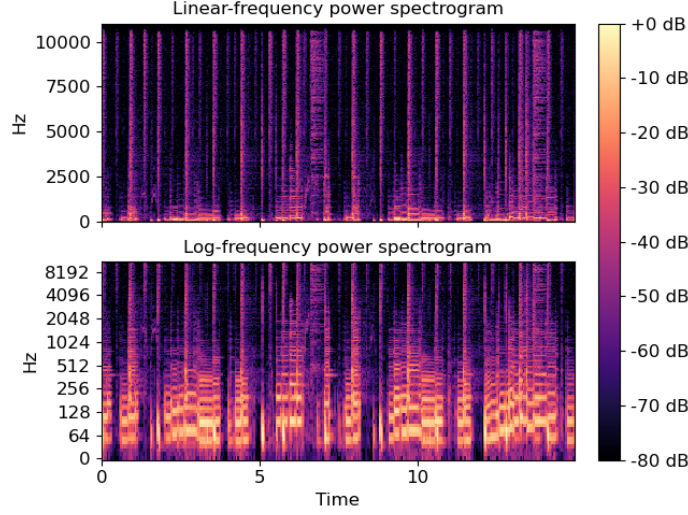


Figura 4.10: Espectrograma estándar arriba y espectrograma de Mel abajo.

4.1.3. Mezcla de señales y separación de fuentes

En esta subsección se comentarán las diferentes metodologías de mezcla aprendidas para entender el funcionamiento de la mezcla de señales sonoras.

4.1.3.1. Mezclas lineales

Una mezcla lineal de señales se produce cuando varias fuentes $s_i(t)$ se suman con diferentes pesos a_i o coeficientes:

$$x(t) = \sum_{i=1}^N a_i \cdot s_i(t)$$

En una producción musical estos coeficientes pueden representar de manera simplificada las diferencias de nivel entre las pistas producidas por la distancia entre la fuente y el micrófono, la sensibilidad del micrófono, la aplicación de ganancia a la señal o los ajustes realizados durante la mezcla.

Sin embargo una mezcla de música real no puede describirse únicamente mediante la suma de fuentes multiplicadas por ganancias constantes. En una señal estéreo cada fuente puede distribuirse de forma diferente entre los canales izquierdo y derecho mediante el panning. Además la propagación del sonido en una sala real introduce retardos y reverberación, a lo que pueden añadirse

efectos de producción como ecualización o compresión, entre otros. Por tanto, en un modelo realista, cada fuente puede experimentar transformaciones diferentes antes de incorporarse a la mezcla lo que demuestra poca relación con la realidad de esta interpretación.

4.1.3.2. Mezclas coherentes e incoherentes

Las mezclas coherentes son aquellas que están correlacionadas o que son dependientes en tiempo y frecuencia, fase y estructura. Por ejemplo, dos guitarras tocando el mismo *riff*, o un coro cantando al unísono.

Por otro lado las mezclas incoherentes son aquellas que no presentan relación de tiempo y frecuencia. Un ejemplo claro de mezcla incoherente podría ser una batería tocada por encima de una progresión de acordes pero que en el análisis temporal los golpes de la batería suenan a destiempo en relación con los acordes de la progresión.

4.1.4. Representaciones digitales

En esta sección se explicarán las diferentes representaciones digitales del sonido.

4.1.4.1. Formatos del audio

Existen varias formas de guardar una representación digital del sonido:

- **Formatos sin compresión:** *WAV*, *AIFF*. Formatos que guardan directamente las muestras de audio codificadas, ofrecen mayor fidelidad al original pero también generan archivos más pesados. *WAV* es el formato utilizado como representación digital del sonido a lo largo de todo el proyecto.
- **Otros formatos:** como formatos comprimidos sin pérdida (*FLAC*, *ALAC*... reducen el tamaño sin alterar información) o formatos comprimidos con pérdida (*MP3*, *AAC*, *OGG*... descartan información que el oído humano no percibe)

CAPÍTULO

Estado del arte 5



En el ámbito de la inteligencia artificial la separación de fuentes musicales es un campo que ha experimentado gran evolución desde sus primeros acercamientos en 1973 , donde se utilizaba un *vocoder homomórfico*[5] para separar la función de excitación de la respuesta al impulso del tracto vocal en voces. A lo largo de su evolución, los métodos de separación de fuentes musicales se han desarrollado a partir de distintos paradigmas metodológicos, entre los que destacan el procesamiento de señales, el modelado o la modulación del audio y del discurso, y los enfoques probabilísticos o guiados por datos. Esta clasificación general servirá como marco para estructurar el presente capítulo.

Primeramente en el grupo de procesado de señales se persigue obtener un filtrado de la mezcla. Esto se logra comunmente aplicando un filtro de pasa baja, pasa alta o de banda a la mezcla completa. También engloba esta rama la generación de máscaras de ganancia, que son máscaras de valores en rango [0-1] a aplicar sobre una mezcla y que devuelven una cadena de valores correspondientes a la representación tiempo-frecuencia de la fuente buscada.

En segundo lugar, en la modulación de audio o modulación de discurso, se estudia la predictibilidad del comportamiento de las señales haciendo diferenciación entre *señales sinusoidales* o tonos puros y *ruido*, siendo ambos los extremos en una línea de predictibilidad. Por un lado el comportamiento de las señales sinusoidales puede ser predicho en un futuro a partir de muestreos previos, mientras que por otro, el *ruido blanco*, que se define como una señal impredecible a futuro y su espectrograma presenta energía constante en todo el rango frecuencial. Dentro de esta categoría se presentan varias herramientas como por ejemplo *cepstrum* [6] , con aplicaciones de estimación de frecuencias fundamentales , derivación de frecuencias cepstrales , y filtrado de señal.

En los acercamientos probabilísticos, se diferencia el *Modelo oculto de Markov*, modelo específico de una red bayesiana que precisamente tuvo su primer uso en reconocimiento de habla [7] y el *modelo de mezcla Gaussiano* que da una forma de aproximar una distribución como una suma ponderada de *gaussianos*

[8] , útil para crear máscaras.

5.1. Métodos clásicos de separación

Dentro de la clasificación general de la introducción de este capítulo, los métodos clásicos de la separación de fuentes pueden organizarse, de manera más específica, en función del elemento que modelan: métodos basados en el modelado de la fuente principal, métodos centrados en el acompañamiento y enfoques conjuntos o probabilísticos, que consideran simultáneamente varias fuentes o sus relaciones estadísticas.

5.1.1. Modelado de la señal principal

En este enfoque se busca modelar la voz o melodía principal como una señal armónica, para, posteriormente, sustraer esa señal de la mezcla original y obtener el acompañamiento [5]. La voz cantada se produce por vibración de cuerdas vocales, y suele presentar frecuencia fundamental y armónicos, por lo que los métodos más clásicos intentaban estimar el *pitch* y reconstruir o filtrar la señal principal. Dentro de la bibliografía se engloba esta familia de métodos en métodos basados en la hipótesis de armonicidad [5]: primero se estima la frecuencia fundamental de la señal principal y después se obtiene una separación por resíntesis, que consiste en aproximar la fuente mediante los parámetros estimados de su modelo armónico, es decir re generar la pista, o filtrado, donde se modifica la mezcla original mediante un filtro o máscara que conserva los elementos objetivo y atenúa el resto.

El procedimiento más común suele ser:

1. Se parte de la hipótesis de que la señal principal es armónica, que es una señal cuya energía se concentra en una frecuencia fundamental f_0 y en múltiplos enteros de esta dentro del espectro frecuencial, que son los armónicos.
2. Se estima la frecuencia fundamental.
3. Se localizan los armónicos.
4. Se reconstruye o filtra la fuente principal.

El método se realiza sobre el espectro de la mezcla identificando en secciones la frecuencia fundamental con métodos como la detección de picos, por ejemplo. Posteriormente se selecciona como fundamental la frecuencia que mejor explica la periodicidad observada y a partir de ella se encuentran los armónicos en torno a sus múltiplos enteros. Este método presenta algunas limitaciones:

- El método falla si la voz no es puramente armónica, cosa que ocurre con frecuencia por factores como que las vocales introducen vibraciones periódicas de los pliegues vocales o que las consonantes sordas y la respiración introducen componentes aperiódicos o ruidosos.

- Dificultan el método los casos en los que los instrumentos son armónicos o la voz pueda estar muy mezclada con reverberación o efectos.

5.1.2. Modelado del acompañamiento

Este enfoque es un poco la idea opuesta al anterior: se modela el acompañamiento para posteriormente substrarlo de la mezcla original y obtener la voz. Esto se realiza normalmente transformando la mezcla en un espectrograma e identificando en el patrones repetitivos.

El acompañamiento suele ser más redundante o repetitivo que la señal principal [9]: patrones rítmicos, armónicos, o estructurales que suelen repetirse en el tiempo. Esta redundancia permite modelarlo y separarlo, muchos acercamientos clásicos basados en el modelado del acompañamiento toman ventaja de esta característica: la redundancia, se aprovechan componentes de bajo rango, voz dispersa, y repeticiones internas del acompañamiento.

Entre las limitaciones de este enfoque se encuentran que: no todo acompañamiento es repetitivo [10], la existencia de canciones con cambios estructurales fuertes, existencia de elementos solistas o improvisados que producen imprevisibilidad o presencia de repetibilidad en el elemento vocal.

5.1.3. Acercamientos conjuntos y probabilísticos

A diferencia de los métodos anteriores, estos enfoques no modelan únicamente la voz o el acompañamiento para obtener la otra fuente por sustracción, sino que representan ambas fuentes de manera conjunta mediante modelos estadísticos.

Entre los modelos que históricamente han presentado mejor desempeño se encuentran:

- Modelo oculto de Markov [11] (HMM): es un modelo estadístico que modela una secuencia mediante estados ocultos cuya evolución depende del estado anterior. En separación de fuentes estos estados pueden representar diferentes configuraciones de la voz y el acompañamiento, mientras que las observaciones corresponden a características espectrales extraídas de la mezcla. Así, el modelo permite estimar la secuencia de estados más probable y aprovechar la continuidad temporal de las fuentes.

- Modelo de mezcla Gaussiana [8] (GMM): modela las características espectrales de cada fuente mediante una combinación de distribuciones gaussianas. Para cada observación de la mezcla calcula la probabilidad de que haya sido generada bien por la voz o bien por el acompañamiento. Estas probabilidades se emplean para construir una máscara tiempo-frecuencia y estimar la contribución de cada fuente.

Ambos modelos se utilizan para asignar valores a celdas tiempo-frecuencia de fuentes. Modelan estados o distribuciones que pueden ayudar a estimar fuentes o máscaras y no siempre asignan directamente celdas TF.

Entre las limitaciones de estos acercamientos encontramos:

- Requerimiento de hipótesis explícitas previas tratando de dar explicación al comportamiento de las fuentes
- Posible dificultad de ajustar los parámetros que requieren ambos modelos, especialmente porque se suele disponer de pocos datos.
- La posible falta de correspondencia entre el modelo estadístico elegido y la complejidad de la música real, lo que reduce la calidad de la separación.

5.2. Enfoques basados en aprendizaje profundo

El aprendizaje profundo ha adquirido gran relevancia en la separación de fuentes gracias al aumento de la capacidad computacional de las últimas dos décadas y a la disponibilidad de conjuntos de datos que contienen mezclas y fuentes aisladas. Los modelos pueden operar directamente sobre la forma de onda o sobre representaciones tiempo-frecuencia, como el espectrograma, aunque el procesamiento directo de la onda permite conservar toda la información de la señal, incluida la fase, pero requiere trabajar con secuencias de gran tamaño lo que incrementa la complejidad y coste computacional.

La mayoría de estos sistemas se basan en la tecnología de las redes neuronales. Estos paradigmas computacionales aprenden durante una fase de entrenamiento una fase de transformaciones para relacionar la mezcla de entrada con las fuentes que la componen. A partir de ejemplos formados por conjuntos mezcla-pistas, la red aprende las características asociadas a cada fuente y posteriormente puede estimarlas en grabaciones que no han sido utilizadas durante el entrenamiento.

5.2.1. Modelos basados en espectrograma

Muchos modelos neuronales toman como entrada una magnitud STFT o un espectrograma, y producen como salida una máscara o una magnitud estimada.

Estos modelos transforman inicialmente la mezcla al dominio tiempo-frecuencia mediante la Transformada en Tiempo Corto de Fourier. A partir de ahí, la magnitud del espectrograma se introduce en una red neuronal que estima una máscara para cada fuente. Finalmente cada máscara se aplica al espectrograma de la mezcla y utilizando normalmente su fase se reconstruye la señal temporal mediante la transformada inversa.

Ejemplos a tener en cuenta de estos métodos son:

- **Open-Unmix**[12]: implementación abierta y reproducible basada en este enfoque. El modelo procesa la magnitud del espectrograma y estima las fuentes correspondientes a la voz, la batería, el bajo y el resto del acompañamiento. En la evaluación presentada por sus autores sobre MUSDB18, obtuvo resultados comparables a los mejores sistemas

analizados, por lo que fue propuesto como modelo de referencia para la investigación en separación musical.

- **Spleeter** [13]: herramienta desarrollada por *Deezer* que proporciona modelos preentrenados basados en arquitecturas de tipo U-Net. A partir del espectrograma de la mezcla, estima máscaras para realizar separaciones en dos, cuatro o cinco fuentes. Entre sus principales características se encuentran su rapidez de procesamiento y la disponibilidad de modelos preparados para su utilización.

Open-Unmix [12].

5.2.2. Modelos *waveforms* e híbridos

Los modelos basados en forma de onda procesan directamente las muestras de audio y generan una señal temporal para cada fuente sin utilizar necesariamente un espectrograma como representación intermedia. Un ejemplo de este enfoque es **Demucs**[14], que sigue este enfoque mediante una estructura neuronal codificador-decodificador.

La evolución moderna en este campo no se limita solamente a utilización de muestras de audio en lugar de un espectrograma, extiende también a modelos que combinan ambas representaciones, ondas y espectrogramas. Por ejemplo, **Hibryd Demucs** [15]: combina dos ramas de procesamiento, una trabaja directamente sobre la forma de onda y la otra sobre el espectrograma. La información obtenida en ambos dominios se utiliza en conjunto para estimar las fuentes separadas, permitiendo aprovechar tanto la estructura temporal como la frecuencial de la música.

5.2.3. Conjunto de datos y evaluación

MUSDB 18 es un conjunto de datos y *benchmark* de referencia para el desarrollo y la evaluación de sistemas de separación de fuentes musicales. Contiene 150 canciones completas, con una duración total aproximada de diez horas, divididas en 100 canciones de entrenamiento y 50 de prueba. Para cada canción se proporciona la mezcla y cuatro fuentes aisladas: voz, batería, bajo y resto del acompañamiento [16].

5.3. Herramientas y tecnologías actuales

Actualmente gracias al gran desarrollo de las capacidades de computo y el desarrollo de bases de datos amplias, existen tecnologías que dan muy buenos resultados, en el contexto de los métodos anteriormente mencionados, los métodos que aparecen y reinantes de la actualidad corresponderían a la categoría de modelos guiados por datos.

Algunos de los más interesantes:

- **Open-Unmix:** implementación de red neuronal referente en el campo de la separación musical. Provee de modelos preparados para su uso que permite separación de pistas pop en cuatro fuentes: vocales, baterías, bajo y resto. Se trata de un modelo abierto basado en deep learning y que permite reproducibilidad [12].
- **Spleeter:** conjunto de modelos pre-entrenados de separación de fuentes en pista musicales. Basados en redes neuronales convolucionales con salto de conexiones (que se traducen en codificadores-decodificadores llamados *U-Nets*) de doce capas, utilizando función de pérdida *L1-norm*, entrenados con bases de datos internas de *Deezer* y con espectrogramas de la mezcla como entrada [13].
- **Demus / Hybrid Demucs:** modelo avanzado que, en función del tipo de pista de entrada, toma como entrada su espectrograma, su onda, o ambos. De los estados más modernos en el campo [15].

Además, mencionar también **MUSDB18** que es un dataset/benchmark de referencia que contiene 150 conjuntos de mezcla y cuatro fuentes comprendiendo bajo, batería, voz y resto. Permite comparar sistemas y está dividido en entrenamiento y test. Ha sido el *dataset* utilizado en este trabajo de fin de grado.

CAPÍTULO Desarrollo 6



En este capítulo se explicará el desarrollo del proyecto. Se ha dividido el capítulo en dos secciones correspondientes al desarrollo del modelo de separación de fuentes y al desarrollo de la interfaz gráfica del sistema.

6.1. Modelo de separación de fuentes

Tras un estudio en profundidad de los métodos comúnmente utilizados para el objetivo que nos concierne se observó que existe un gran abanico de posibilidades. Se termina por optar por un modelo de inferencia de máscaras ya que presentaba el recurso de un tutorial de uso en formato web junto a la definición técnica del modelo. Algunas particularidades del modelo elegido son las siguientes:

- El modelo no produce directamente una señal acústica, si no que predice una máscara tiempo-frecuencia que se aplica sobre la mezcla filtrando la señal, intentando así proporcionar una fuente como salida del filtrado.
- La estimación que proporciona la salida no es una señal, es el valor de la magnitud de la señal en cada punto del espectrograma.
- En el entrenamiento se comparan magnitudes estimadas hasta el momento por el modelo con magnitudes objetivo proporcionadas por la base de datos.
- Tras el proceso de inferencia de la máscara se reconstruye el audio usando la fase de la mezcla.

6.1.1. Base de datos

Para la base de datos se eligió *MUSDB18*[16].

Se trata de una relación de 150 pistas musicales de diferentes géneros y sus respectivos elementos musicales aislados en cuatro grupos: batería, bajo, vocales y otros.

En la web de presentación de la base de datos a la cual se puede acceder desde el link proporcionado en la bibliografía se encuentran dos conjuntos diferentes: *MUSDB18* y *MUSDB18-HQ*. Estos dos grupos contienen la misma relación de canciones y pistas separadas, comprimidas y descomprimidas respectivamente. Si bien el segundo grupo puede ofrecer más datos el conjunto descomprimido contiene mucha más información que el comprimido, por lo que por cuestiones de optimización de tiempos de entrenamiento se optó por el conjunto comprimido.

6.1.2. Datos de entrenamiento

El conjunto elegido e introducido en la sección anterior se separa en dos carpetas, *train* de 100 canciones y *test* con 50 canciones. La documentación indica que los enfoques supervisados deben ser entrenados en el conjunto de entrenamiento y probados en los conjuntos de test y de entrenamiento, y, además, que todas las pistas son estereofónicas y codificadas en 44.1kHz.

Para cada pista se tiene:

- *mixture.wav*: la mezcla original.
- *vocals.wav*: pista para las vocales.
- *drums.wav*: pista para la batería.
- *bass.wav*: pista para el bajo.
- *other.wav*: pista para elementos extra.

6.1.3. Funciones de pérdida

En el proyecto se comparan diferentes funciones de pérdida con el objetivo de analizar cómo afecta la medición del error al entrenamiento del modelo de separación.

Las funciones de pérdida son parte crucial del aprendizaje automático. El término refiere a una función matemática que estima el error existente entre la estimación y el objetivo. Dependiendo de la función de pérdida elegida, el valor numérico de salida prioriza dar un tipo de información u otra sobre la diferencia entre la fuente estimada y la de referencia. En la Tabla 6.1 se pueden ver las funciones de pérdida principales que se han valorado en el proyecto, su nombre, la información que se enfocan en dar y la expresión matemática que las define. En la tabla: Y es la magnitud objetivo de la fuente, $\hat{Y}$ la magnitud estimada, X es la magnitud de la mezcla, $\hat{M}$ la máscara estimada, N el número

de elementos comparados y ϵ la constante para evitar divisiones o logaritmos de cero.

Función de pérdida	Comentario	Expresión matemática
L1	Penaliza el error absoluto entre la magnitud estimada y la magnitud objetivo. Es una pérdida simple y robusta frente a errores extremos.	$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N \hat{Y}_i - Y_i $
L2	Penaliza el error cuadrático entre la estimación y la referencia. Da más peso a errores grandes que la pérdida L1.	$\mathcal{L}_{L2} = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i - Y_i)^2$
L1 frecuencia	Variante de L1 aplicada sobre una reorganización frecuencial de los tensores, tratando cada fuente del batch como una muestra independiente.	$\mathcal{L}_{L1freq} = \frac{1}{N} \sum_{i=1}^N \hat{Y}_i^{freq} - Y_i^{freq} $
L2 frecuencia	Variante de L2 aplicada sobre la representación frecuencial. Mantiene la penalización cuadrática tras reformular las dimensiones de los tensores.	$\mathcal{L}_{L2freq} = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i^{freq} - Y_i^{freq})^2$
Log L1	Calcula el error absoluto y lo expresa en una escala logarítmica. Reduce el efecto del rango dinámico de las magnitudes.	$\mathcal{L}_{logL1} = \frac{1}{B} \sum_{b=1}^B \log_{10} \left(\sum_i \hat{Y}_{b,i} - Y_{b,i} + \epsilon \right)$
Log L2	Calcula el error cuadrático y lo transforma a escala logarítmica. Penaliza errores grandes, pero con una escala comprimida.	$\mathcal{L}_{logL2} = \frac{1}{B} \sum_{b=1}^B \log_{10} \left(\sum_i (\hat{Y}_{b,i} - Y_{b,i})^2 + \epsilon \right)$
Log magnitude	Compara la magnitud estimada y la magnitud objetivo después de aplicar una transformación logarítmica. Permite trabajar con diferencias relativas.	$\mathcal{L}_{logMag} = \frac{1}{N} \sum_{i=1}^N \left \log_{10}(\hat{Y}_i + \epsilon) - \log_{10}(Y_i + \epsilon) \right $
Log-compressed L2	Calcula una pérdida cuadrática entre magnitudes log-comprimidas. Su objetivo es reducir el rango dinámico antes de comparar estimación y referencia.	$\mathcal{L}_{logCompL2} = \frac{1}{N} \sum_{i=1}^N \left(\log(\hat{Y}_i + \epsilon) - \log(Y_i + \epsilon) \right)^2$
LPSA	En la implementación utilizada, compara espectros de potencia en escala logarítmica. Permite trabajar con diferencias relativas de energía espectral.	$\mathcal{L}_{LPSA} = \frac{1}{N} \sum_{i=1}^N \left(\log(\hat{Y}_i^2 + \epsilon) - \log(Y_i^2 + \epsilon) \right)^2$
Mask L1	Compara la máscara estimada con una máscara ideal obtenida a partir de las magnitudes objetivo de las fuentes.	$M_i^{ideal} = \frac{Y_i}{\sum_s Y_{i,s} + \epsilon}, \quad \mathcal{L}_{mask} = \frac{1}{N} \sum_{i=1}^N \hat{M}_i - M_i^{ideal} $

Tabla 6.1: Funciones de pérdida principales consideradas durante el entrenamiento.

Además del conjunto de funciones de pérdida principal, se han valorado también funciones de pérdida más avanzadas y en dos casos experimentales:

Función de pérdida	Comentario	Expresión matemática
LPSA phase	Variante que incorpora información de fase de la mezcla y de la fuente objetivo. Requiere disponer de las fases θ_X y θ_Y .	$Y_i^{PSA} = Y_i \cos(\theta_{X,i} - \theta_{Y,i}), \quad \mathcal{L}_{LPSAphase} = \frac{1}{N} \sum_{i=1}^N (\hat{Y}_i - Y_i^{PSA})^2$
LMRS	Pérdida basada en la relación logarítmica entre fuente y mezcla. Compara ratios espectrales en lugar de magnitudes absolutas.	$\mathcal{L}_{LMRS} = \frac{1}{N} \sum_{i=1}^N \left(\log \frac{Y_i + \epsilon}{X_i + \epsilon} - \log \frac{\hat{Y}_i + \epsilon}{X_i + \epsilon} \right)^2$
L-MRS	Pérdida multirresolución calculada tras reconstruir señales temporales. Combina términos espectrales en distintas resoluciones STFT.	$\mathcal{L}_{MRS} = \frac{1}{R} \sum_{r=1}^R (SC_r(\hat{y}, y) + \mathcal{L}_{logMag,r}(\hat{y}, y))$
Deep-feature	Compara activaciones internas extraídas por una red auxiliar a partir de los espectrogramas estimado y objetivo.	$\mathcal{L}_{DF} = \sum_{j \in J} \ \phi_j(\hat{Y}) - \phi_j(Y)\ _1$
Deep-feature-EMD	Variante experimental que compara distribuciones de activaciones profundas mediante una aproximación regularizada de la distancia Earth Mover.	$\mathcal{L}_{DF-EMD} = \sum_{j \in J} \text{Sinkhorn}(\mu_{\phi_j(\hat{Y})}, \mu_{\phi_j(Y)})$

Tabla 6.2: Funciones de pérdida avanzadas y experimentales consideradas durante el desarrollo.

La función de pérdida *Deep-feature-EMD*, propuesta por el tutor de este proyecto, hace uso de una distancia tipo *Sinkhorn/EMD* sobre las características extraídas por una CNN auxiliar. Define la parte mas experimental del proyecto.

Cuando introduces un espectrograma en una CNN cada capa convolucional aplica diferentes filtros que responden a patrones específicos presentes en la entrada. Las salidas producidas por esos filtros se llaman activaciones o mapas de características. Las primeras capas pueden responder a características relativamente simples mientras que capas posteriores pueden representar combinaciones mas complejas de esos patrones.

La pérdida DeepFeature-EMD compara la estimación y el objetivo no directamente en el dominio de magnitud, sino en un espacio de activaciones. Para cada capa considerada, las activaciones se aplanan y se interpretan como distribuciones[17]. La discrepancia entre ambas distribuciones se mide mediante una distancia Sinkhorn, que es una derivación de la Earth Mover's Distance.

De este modo, la función de pérdida evalúa la similitud entre ambas señales en un espacio de características aprendido por una CNN auxiliar, en lugar de la comparación entre magnitudes espectrales que presentan el resto de funciones de pérdida evaluadas. Esta función constituye una de las propuestas experimentales del proyecto y fue incorporada a partir de una propuesta del tutor. El extractor CNN auxiliar no está entrenado en música, por lo que la pérdida se considera experimental y no necesariamente correlaciona con calidad perceptual.

6.1.4. Arquitectura del modelo

El modelo empleado es una arquitectura recurrente de inferencia de máscaras, basada en una LSTM que recibe la magnitud de la STFT de la mezcla y predice una máscara por fuente. Estas máscaras se aplican sobre la magnitud de la mezcla para obtener las magnitudes estimadas de cada *stem*. En la Figura 6.1 se muestra un ejemplo de una arquitectura LSTM.

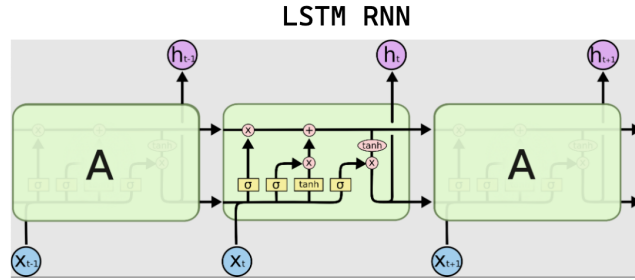


Figura 6.1: Ejemplo de una arquitectura RNN-LSTM [5]

La arquitectura LSTM es comúnmente utilizada en separación de fuentes por su capacidad de retener información, de olvidarla, de análisis secuencial y de mantener contexto pasado y futuro de la información actual. Una LSTM procesa una secuencia paso a paso, normalmente palabras, sonidos o datos en general que se definen a lo largo de un eje temporal.

En el contexto del proyecto el sistema introduce al modelo la interpretación en secuencia temporal de un espectrograma (es decir, la magnitud de la STFT de la entrada), a partir del cuál, la LSTM procesa esta secuencia para estimar máscaras que aplicadas sobre la mezcla original producen una pista fuente. A lo largo del proyecto ha habido modificaciones de los parámetros utilizados y la configuración final se puede ver en la Tabla 6.3.

6.1. Modelo de separación de fuentes

Parámetro	Definición	Valor
Tamaño de ventana STFT	Número de muestras de la onda de entrada sobre las que se aplica la transformada de Fourier de tiempo corto.	512 muestras
Salto STFT	Separación entre una ventana y la consecutiva.	128 muestras
Tipo de ventana	Forma de la ventana utilizada en la STFT.	<code>sqrthann</code>
Número de bins de frecuencia	Número de frecuencias únicas resultantes de una FFT de 512 puntos sobre una señal real.	257
Canales de entrada	Número de canales de audio procesados por la red.	1, mono
Tipo de red recurrente	Red encargada de modelar las dependencias temporales entre espectros consecutivos.	LSTM
Número de capas recurrentes	Número de capas LSTM utilizadas.	1
Tamaño oculto	Número de unidades ocultas por dirección en cada capa LSTM.	50
Bidireccional	Determina si la LSTM analiza tanto el contexto pasado como el futuro.	Sí
Dropout	Probabilidad de desactivar aleatoriamente unidades durante el entrenamiento.	0.0
Activación de salida	Función que transforma las salidas en valores comprendidos entre cero y uno.	Sigmoid
Salidas en 2 stems	Fuentes estimadas en la configuración de dos stems.	vocals, accompaniment
Salidas en 4 stems	Fuentes estimadas en la configuración de cuatro stems.	vocals, bass, drums, other

Tabla 6.3: Configuración arquitectónica de parámetros utilizada en los entrenamientos finales del proyecto.

6.1.4.1. Representación de los datos

6.1.4.1.1. Preprocesado del conjunto de datos En esta sección se explicará el procesado que se aplica para ir desde los archivos de audio de entrada hasta los *batches* con un formato apto para el entrenamiento.

- **1. Construcción de escenarios:** como ya se ha comentado en secciones anteriores, el modelo contempla dos escenarios de salida: cuatro y dos pistas. Partiendo de ello, la relación de mezcla-fuente de entrada debe de tener un formato acorde a la salida que se espera. Para ello se diferencia:
 - *2 stems: vocales, acompañamiento (batería + bajo + otros).* Se mantiene la fuente de las vocales de la base de datos y se agrupa el resto como acompañamiento.

- *4 stems: batería, bajo, vocales, otros.* Se usan las cuatro fuentes originales de la base de datos.

- **2. Conversión a dominio espectral:** Se aplica la misma transformada de Fourier a la mezcla y a las fuentes. A partir de ahí se extraen magnitudes para entrenamiento y se guarda la fase de la mezcla para la posterior reconstrucción de la estimación.

La mezcla de audio se transforma al dominio tiempo-frecuencia mediante la STFT, obteniendo un espectrograma.

Después se extrae la magnitud del espectrograma, que constituye la entrada principal del modelo, mientras que la fase se conserva para la reconstrucción posterior del audio. Finalmente la magnitud se organiza en forma de tensor, se convierte a escala de decibelios y se normaliza antes de introducirse a la red neuronal recurrente con el objetivo de reducir su rango dinámico y facilitar el entrenamiento.

- **3. Preparación de *batches*:** cada *batch* contiene al menos:

- *mix_magnitude*: magnitud de la mezcla.
- *source_magnitudes*: magnitudes de las fuentes.

Además algunas funciones de pérdida avanzadas requieren información de fase, por lo que en el preprocesado se crea un tipo especial de entrada para ellas que contiene:

- *mixture_phase*: fase de la mezcla.
- *source_phase*: fase de las fuentes.

La última versión del proceso de entrenamiento comprueba la presencia de estos dos últimos para las funciones que las piden, y usa *source_magnitudes* como entrada principal durante el entrenamiento.

- **4. División train - validation - test:**

- *train/validation*: usado durante entrenamiento.
- *representative_test*: conjunto fijo usado solo para evaluación final.

División 60-40 , 60 % del conjunto de entrenamiento y 40 % de validación.

- **5. Normalización de canales:** En la documentación[18] de la arquitectura del modelo se recomienda un solo canal de audio como entrada por lo que los datos de entrenamiento se normalizan a un solo canal. Además esto reduce el coste computacional y simplifica el proceso de comparación entre fuentes.

6.1.4.1.2. Entrada al modelo y objetivos La entrada principal al modelo se puede resumir como en el esquema de la Figura 6.2:

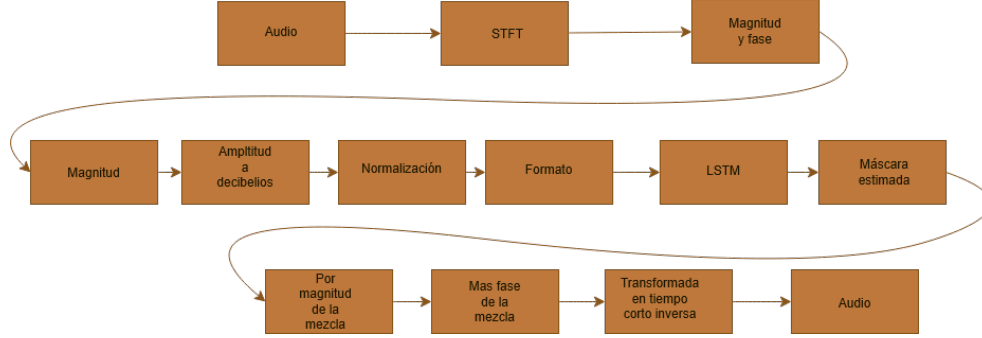


Figura 6.2: Esquema del proceso de inserción de datos al modelo de inferencia de máscaras.

La estructura de entrada al modelo o *input* es un vector que contiene:

- B : tamaño del *batch*, es decir, número de ejemplos procesados simultáneamente.
- T : número de frames temporales del espectrograma.
- F : número de bins de frecuencia obtenidos mediante la STFT.

$$[B, T, F]$$

Y la estructura de salida o *targets* es otro vector que contiene:

$$[B, T, F, C, S]$$

Donde:

- B : tamaño del batch.
- T : número de frames temporales.
- F : número de bins de frecuencia.
- C : número de canales de audio. En este trabajo es una constante $C = 1$, ya que las señales de entrada se transforman a mono por indicación de la documentación como ya se ha comentado en la sección anterior.

- S : número de fuentes o *stems* a separar. En este trabajo puede ser $S = 2$ para voz y acompañamiento, o $S = 4$ para voz, bajo, batería y resto.

La salida del modelo no genera audio directamente, la salida son máscaras tiempo-frecuencia. Posteriormente, se aplica la máscara sobre la magnitud de la mezcla y el resultado corresponde a una estimación de la magnitud de la fuente. Luego para reconstruir el audio se utiliza la fase de la pista original. Esto es una limitación importante, ya que es una estimación. La fase de la mezcla original no tiene por qué ser la de las fuentes estimadas, pero a falta del dato de la magnitud para estas últimas, se utiliza la fase de la mezcla de la que se tiene la magnitud lo que permite estimar este dato.

Los targets (objetivos) son las magnitudes de las fuentes reales, separados en los grupos de dos y cuatro fuentes que se han explicado en la sección anterior de este capítulo.

6.1.4.1.3. Transformaciones personalizadas de los datos En esta sección se comentarán las transformaciones (TFMs) aplicadas al audio/tensores antes del entrenamiento:

El flujo de procesamiento avanza de la siguiente manera:

Audio $\rightarrow$ STFT $\rightarrow$ Magnitud y fase

Magnitud $\rightarrow$ AmplitudeToDB $\rightarrow$ BatchNorm $\rightarrow [B, T, F] \rightarrow$ LSTM $\rightarrow$ Máscara estimada

Máscara estimada $\times$ Magnitud de la mezcla + Fase de la mezcla $\rightarrow$ iSTFT

Donde cada transformación es:

- **Transformaciones espectrales.**

En primer lugar, la señal de audio se transforma desde el dominio temporal al dominio tiempo-frecuencia mediante la STFT (*Short-Time Fourier Transform*). Esta transformación divide la señal en ventanas temporales sucesivas y aplica una transformada de Fourier a cada una de ellas, obteniendo una representación compleja que indica el contenido frecuencial de la señal en cada instante temporal.

A partir de la STFT se separan dos componentes principales:

- **Magnitud:** corresponde al módulo de cada coeficiente complejo de la STFT. Representa la intensidad relativa de cada componente frecuencial en cada frame temporal. En este trabajo, la magnitud de la mezcla se utiliza como entrada principal del modelo, mientras que las magnitudes de las fuentes reales se utilizan como objetivos durante el entrenamiento.

- **Fase:** corresponde al ángulo de cada coeficiente complejo de la STFT. Contiene información sobre el desplazamiento temporal relativo de las componentes frecuenciales. El modelo no estima directamente la fase de cada fuente, por lo que durante la reconstrucción se reutiliza la fase conocida de la mezcla como aproximación.

En el caso de separación en dos fuentes, las fuentes del dataset pueden agruparse para formar los objetivos vocales y acompañamiento. En este caso, el acompañamiento se obtiene a partir de la combinación de las fuentes no vocales, como bajo, batería y resto.

■ **Transformaciones internas del modelo.**

Una vez obtenida la magnitud de la mezcla, se aplican varias transformaciones internas antes de introducirla en la red recurrente:

- **AmplitudeToDB:** convierte la magnitud lineal a una escala logarítmica expresada en decibelios. Esta transformación comprime el rango dinámico de la señal, reduciendo la diferencia entre componentes de muy alta y muy baja energía. Esto facilita que el modelo trabaje con valores más estables durante el entrenamiento.
- **BatchNorm:** la normalización por lotes normaliza las activaciones utilizando estadísticas calculadas sobre el batch. En este trabajo se aplica después de la conversión a decibelios y antes de la LSTM, con el objetivo de estabilizar el entrenamiento y facilitar la convergencia del modelo.
- **Reordenación dimensional:** las operaciones de carga de datos, cálculo de STFT y procesamiento con redes recurrentes pueden requerir distintas ordenaciones de los ejes del tensor. Por ello, antes de introducir los datos en la LSTM, la magnitud se adapta al formato $[B, T, F]$. De esta forma, el espectrograma se interpreta como una secuencia temporal de vectores frecuenciales.

La LSTM procesa esta secuencia temporal y produce una representación a partir de la cual se estiman máscaras tiempo-frecuencia para cada fuente. Estas máscaras se aplican sobre la magnitud de la mezcla para obtener las magnitudes estimadas de los distintos *stems*.

■ **Transformaciones para pérdidas concretas.**

Algunas funciones de pérdida requieren adaptar la forma de los tensores antes de calcular el error entre la estimación y el objetivo. Estas transformaciones no modifican el contenido de la señal, sino la organización de sus dimensiones para adecuarlas al tipo de comparación que realiza cada pérdida.

- **Pérdidas de frecuencia:** en las pérdidas que comparan directamente representaciones frecuenciales, las estimaciones y los objetivos pueden reformularse al formato $[B \cdot S, F, T]$, donde S es el número de fuentes. De este modo, cada fuente de cada ejemplo del batch se trata como una muestra independiente en el cálculo de la pérdida.
- **Pérdidas *deep-feature*:** en las pérdidas basadas en características profundas, las magnitudes estimadas y objetivo se adaptan al formato $[B \cdot S, 1, F, T]$. Esta forma permite introducir los espectrogramas en una red convolucional auxiliar, donde el eje 1 representa un único canal de entrada y las dimensiones F y T se tratan como una representación bidimensional tiempo-frecuencia.

6.1.5. Entrenamiento

6.1.5.0.1. Resultados preliminares y experimentales Los entrenamientos fueron agrupados por funciones de pérdida, en particular por una estructura de tipos dentro de estas para ayudar a su posterior documentación y evaluación:

- **Pérdidas principales:** $L1$, $L2$, $L1_freq$, $\log L1$, $\log L2$, $\log_mag$, $\log_compressed_l2$, $LPSA$, $mask_L1$.
- **Pérdidas experimentales:** $deep_feature$, $deep_feature_emd$.
- **Pérdidas avanzadas:** l_mrs , $lpsa_phase$.

También existe la división que ya se explicó anteriormente entre dos y cuatro fuentes. El caso de las dos fuentes representa una separación más sencilla y cercana a escenarios de voz y acompañamiento. El caso de cuatro fuentes introduce mayor complejidad, ya que el modelo debe repartir la mezcla entre varias fuentes instrumentales con solapamiento espectral.

A tener en cuenta sobre la comparativa entre funciones de pérdida del proyecto:

- En las primeras versiones del entrenamiento se detectaron problemas de formato, inferencia y evaluación. El proyecto terminó habiéndose realizado tres tandas de entrenamientos diferentes corrigiendo estos errores, de las cuales la última es la definitiva.
- Las pérdidas experimentales basadas en características profundas, tienen mayor coste y mayor incertidumbre, porque dependen de su extractor auxiliar.
- Las pérdidas avanzadas con fase o reconstrucción temporal son mas complejas y, por tanto, conviene añadir control sobre ellas.

6.1.5.0.2. Comparativas En esta sección se presenta una comparativa preliminar del conjunto de funciones presentado junto con sus versiones de dos y cuatro fuentes.

■ **A. Comparación por número de fuentes**

En la literatura del recurso que presenta la arquitectura del modelo utilizado en este proyecto se comentan en múltiples ocasiones las diferencias entre estos dos enfoques al problema. La separación que resulta en el conjunto de dos fuentes, ya visto en las primeras versiones del entrenamiento, es un enfoque más sencillo que permite una "binarización" del problema al reducirse a dos salidas. En contraposición, en el segundo conjunto, donde la separación se extiende a identificar cuatro elementos en el espectro, los elementos que se puedan identificar si el método es válido pueden variar enormemente. Esto dificulta la identificación de los elementos, ya que, un elemento que a lo mejor está en un rango frecuencial concreto en una mezcla, en otra puede habitar en otro no a lo mejor completamente diferente pero sí lo suficiente para que la sensibilidad de la función a estos cambios no dé al modelo la información necesaria para ser capaz de identificar un elemento que varía. Por esto, es esperable que, por lo general, *4 stems* obtenga peores métricas en la fase de evaluación. Identificar y separar dos elementos por los motivos mencionados, resulta en una tarea más sencilla ya que la variabilidad se reduce a dos elementos, mientras que el caso anterior eleva la premisa de la necesidad de sensibilidad en la función al doble de elementos donde puede variar este rango frecuencial.

Por ejemplo, en géneros como la electrónica o el rap que se basan en elementos frecuencialmente bajos (bajo, sub-bajo) se identifica gran superposición entre fuentes como bajo y baterías, es un sonido más "sucio" armónicamente complejo. Por otro lado, por ejemplo, el blues, el pop, o las canciones de autor, presentan mejores desempeños en la separación por sus características de armónicos vocales e instrumentales más limpios [19].

En la Figura 6.3 se pueden observar las métricas de SNR y SDR por género musical en el caso de cuatro fuentes, donde se puede ver que géneros como el rap o la electrónica presentan peores métricas que otros como el pop o el blues que presentan las mejores.

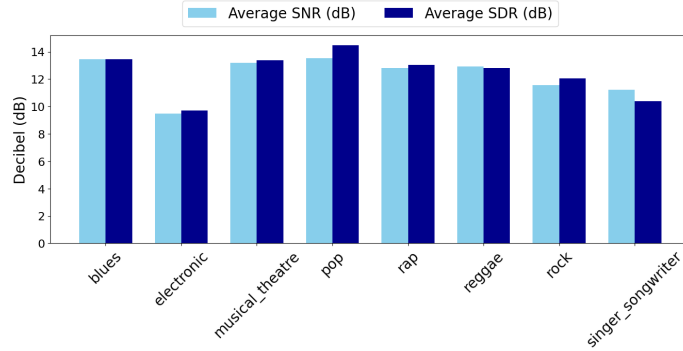


Figura 6.3: Media de métricas SDR y SNR por género musical por género en cuatro fuentes.

■ B. Comparación por familia de pérdida

Se han agrupado las funciones de pérdida por las familias aritméticas a las que pertenecen por su naturaleza:

- Pérdidas de magnitud directa: L1 , L2.
- Pérdidas frecuenciales: L1_freq, L2_freq.
- Pérdidas logarítmicas: logL1, logL2, log_mag, log_compressed_l2.
- Pérdidas informadas por fase o potencia: LPSA, lpsa_phase.
- Pérdidas basadas en máscara: mask_l1
- Pérdidas experimentales: deep_feature , deep_feature_emd

Las distintas familias de pérdidas introducen sesgos de entrenamiento diferentes. Las pérdidas en escala logarítmica trabajan sobre una representación comprimida de la magnitud espectral, ya que la aplicación del logaritmo reduce el rango dinámico del espectrograma [20]. Esto modifica el peso relativo de los errores, evitando que los bins de mayor energía dominen completamente la optimización. Las pérdidas frecuenciales comparan directamente estimaciones y referencias en el dominio tiempo-frecuencia, normalmente mediante distancias punto a punto entre espectrogramas [21]. Por su parte, las pérdidas basadas en máscara emplean como objetivo una máscara ideal o aproximada, guiando al modelo hacia una asignación explícita, aunque generalmente suave, de los bins tiempo-frecuencia a las distintas fuentes [22, 23].

6.1.5.0.3. Entrenamiento V2 Esta versión nace como corrección de la primera versión .

La primera versión, como ya se comentaba en el punto anterior, entrenaba con señales estéreo sin haber cerrado todavía el criterio de canales. Esto hacia

que los tensores de entrada y salida no estuvieran alineados con la arquitectura final planteada.

V2 reentrena los modelos con una configuración más coherente, se entrenan modelos de dos y cuatro fuentes de salida, y sirve para obtener un primer conjunto serio de *checkpoints* (término en inglés para los puntos de guardado del entrenamiento del modelo). Durante la primera evaluación de V2 se detectan nuevos problemas: resultados muy correlacionados con la mezcla y resultados que no superaban el criterio que se estableció de diferencia de correlación mínima con la mezcla original para considerarse un elemento separado.

Finalmente, y por estos motivos, se termina por considerar el conjunto de modelos entrenados como insuficiente, aunque no se descarta completamente y se deja en el repositorio a modo de marca temporal para mostrar la evolución del proyecto.

6.1.5.0.4. Entrenamiento V3 El reentrenamiento número tres corresponde a la versión actual y a la que ha proporcionado los resultados que se muestran en esta memoria. Nace en respuesta a lo visto y aprendido en la versión anterior.

En esta versión se corrige el test que se realizaba por cada modelo ya que se observó que por error se realizaba sobre una fracción aleatoria del conjunto completo. Se añade una *baseline* trivial de mezcla, que establece un criterio de correlación máxima entre resultado y mezcla original, con el objetivo de considerar un elemento como separado y no un archivo equivalente a la mezcla, tratando de dar una medida cuantitativa a la calidad de la separación.

También se unifica la inferencia entre evaluación y despliegue como cambios principales.

Además:

- Se restaura la función de *forward* que se había modificado en la versión anterior bajo sospecha de que ese era el fallo que no permitía una generación de resultados correcta, reduciéndose ahora a:

$$AmplitudeToDB + BatchNorm + RNN + Embedding$$

- Se realiza una separación para organizar los entrenamientos entre pérdidas principales y experimentales.
- Se organiza la salida de los modelos en nuevos directorios.
- Se guarda un sumario de configuración de entrenamiento por modelo, lo que ayuda a hacer un seguimiento a posteriori.
- Se ajusta la planificación por coste final

6.2. Interfaz

Además del proceso de entrenamiento y la evaluación de las funciones de pérdida, se ha implementado una interfaz visual que actúa como herramienta demostrativa y didáctica.

Permite ejecutar modelos entrenados, visualizar resultados y facilita el uso del sistema para el usuario.

6.2.1. Diseño de experiencia de persona usuaria

Según se entra en la interfaz visual, el primer encuentro del usuario con la misma es la pantalla que se puede ver en la Figura 6.4

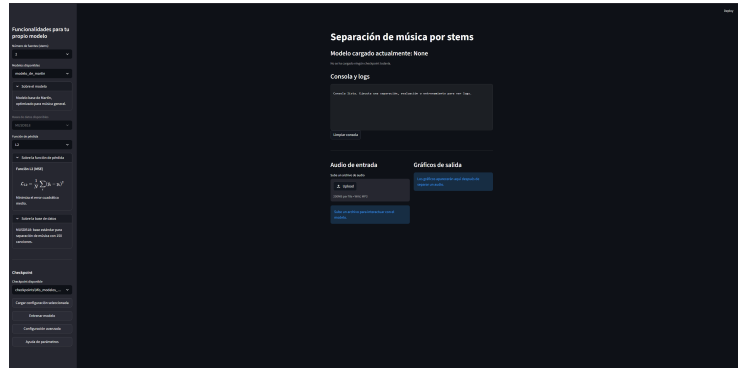


Figura 6.4: Consola y logs de la GUI.

Posteriormente se diferencian los siguientes elementos:

6.2.1.1. Barra lateral de configuración

Se puede apreciar en la imagen de la Figura 6.4.

La barra lateral permite seleccionar el número de fuentes, el modelo disponible, la base de datos, la función de pérdida y el checkpoint que se desea cargar.

6.2.1.2. Zona principal de carga.

Se permite subir un archivo de audio y lanzar el proceso de separación en la sección que se muestra en la Figura 6.5

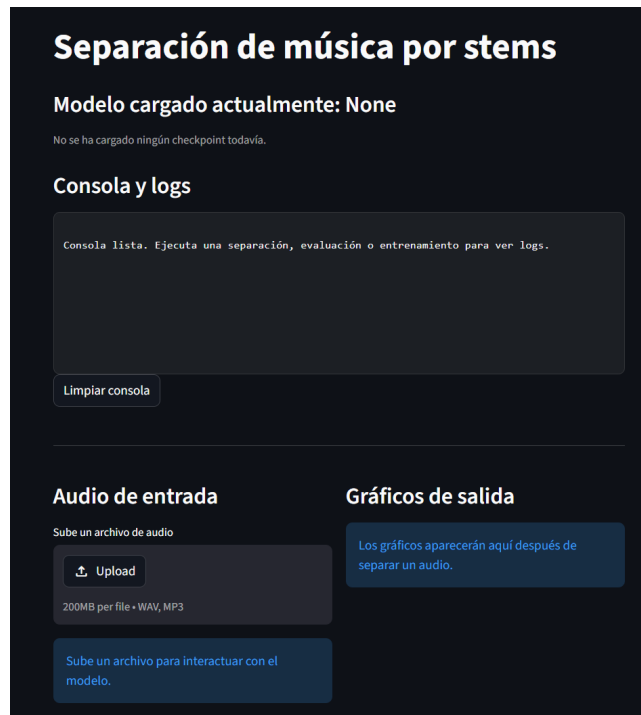


Figura 6.5: Zona principal de carga de la GUI.

6.2.1.3. Consola y logs.

En la sección que se muestra en la Figura 6.6 se imprime en pantalla información sobre la ejecución del sistema, incluyendo mensajes de carga, separación, evaluación y posibles errores.

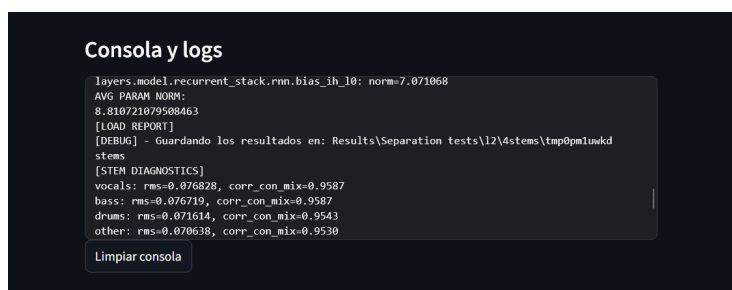
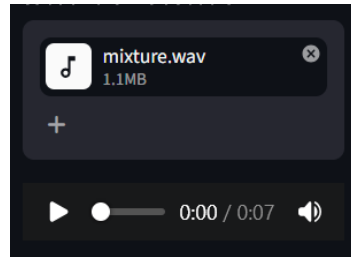


Figura 6.6: Consola y logs de la GUI.

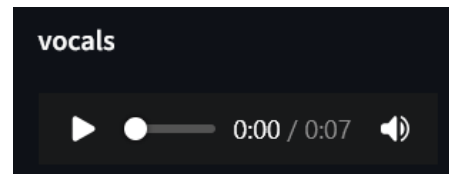
6.2.1.4. Reproductores.

Tras la separación, la interfaz muestra reproductores de audio para la mezcla original y para cada una de las fuentes separadas.

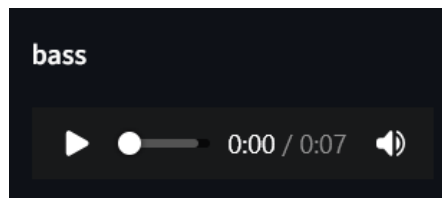
Hay un reproductor para la mezcla original, ver Figura 6.7a y uno para cada fuente separada ya sea la separación de dos o de cuatro fuentes, ver Figuras 6.7b, 6.7c, 6.7d y 6.7e.



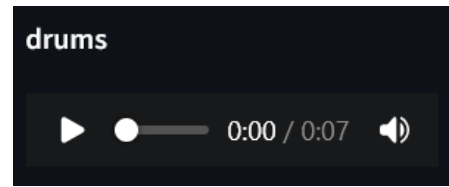
(a) Mezcla original.



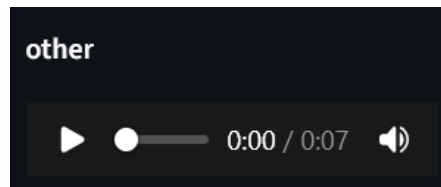
(b) Vocaes.



(c) Bajo.



(d) Batería.



(e) Otros.

Figura 6.7: Reproductores de audio de la interfaz.

6.2.1.5. Gráficos.

La interfaz también permite visualizar representaciones gráficas de las fuentes separadas, como la forma de onda y el espectrograma en las secciones que se muestran en la Figura 6.8.

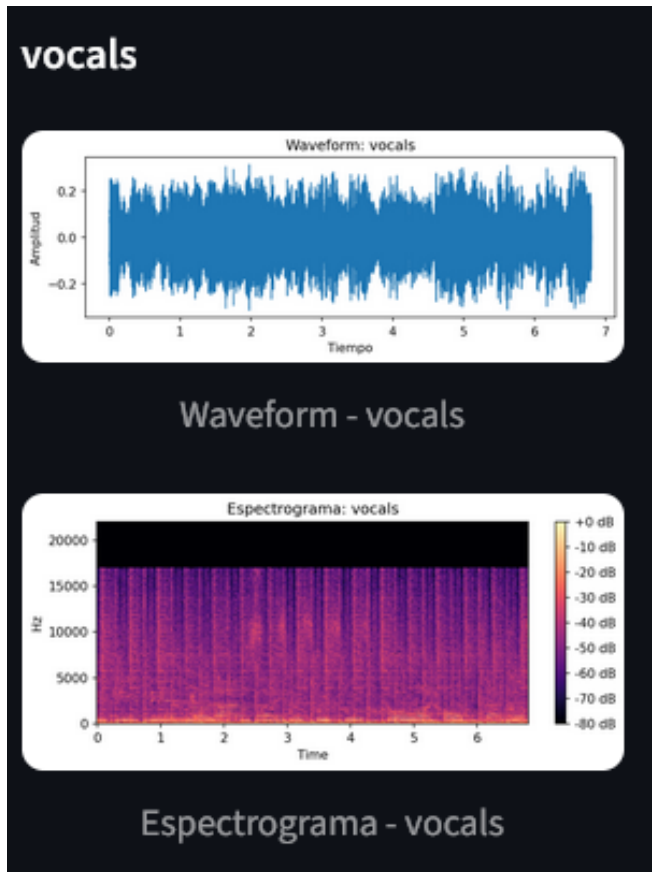
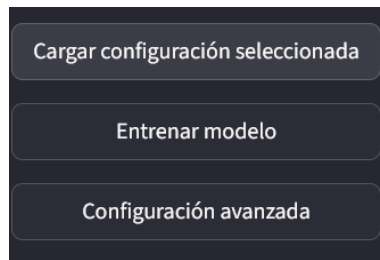


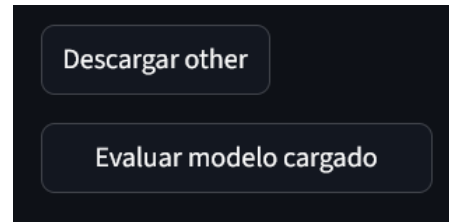
Figura 6.8: Ejemplo de representación gráfica de la fuente vocal.

6.2.1.6. Botones.

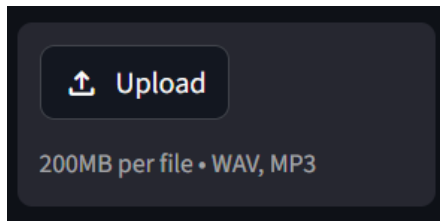
Se incluyen además botones para lanzar la separación, ver Figura 6.9, evaluar el modelo, ver Figura 6.9b, y descargar los audios generados, ver Figura 6.9c.



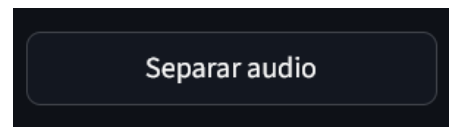
(a) Botón de separación.



(b) Botón de evaluación.



(c) Botón de descarga.



(d) Botón auxiliar.

Figura 6.9: Botones principales de la interfaz.

6.2.1.7. Modificación de parámetros internos del modelo, parámetros de entrenamiento, parámetros de generación de mezclas, de efectos y de la formación de transformadas de Fourier

Existe además una sección de la interfaz que permite modificar los parámetros de formación de las transformadas de Fourier y los parámetros internos del modelo, ver Figura 6.12. Estos parámetros son intrínsecos al modelo y afectan a la separación de fuentes.

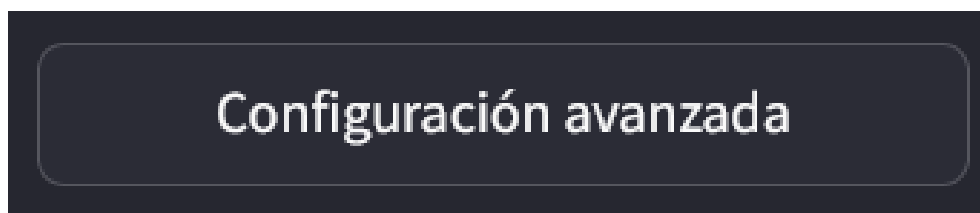


Figura 6.10: Botón de configuración.

El sistema lleva a la persona usuaria a una nueva pantalla donde se pueden modificar variedad de parámetros, ver Figura 6.11.

Entre los parámetros que permite modificar el sistema a la persona usuaria están: los de formación de las transformadas de Fourier, ver Figura 6.12. Además, se pueden modificar los parámetros de entrenamiento de nuevos modelos, ver Figura 6.14, los parámetros de generación de efectos, ver Figura 6.15, y los parámetros de generación de mezclas, ver Figura 6.16.

Configuración avanzada para tu modelo

Modifica los parámetros de entrenamiento y del modelo.

[Ver ayuda de parámetros](#)

Parámetros de la formación de transformadas de Fourier

Tamaño de la ventana STFT
512

Desplazamiento entre ventanas STFT
256

Tipo de ventana utilizada para la STFT
hann

Parámetros intrínsecos modelo

Número de canales de entrada (Mono/Stereo)
1,2

Porcentaje de dropout
0

Funcion de activación para la capa de salida
softmax

Numero de frames para la evaluación del modelo
5

Parámetros de entrenamiento

Learning Rate
0.00100

Batch Size
16

Épocas
100

Tamaño de las épocas
100

Gradient clip
1.00

Weight decay
0.000010

Parámetros de efectos para entrenamiento y de audio

Prob. de aplicar efectos (reverb,pitch,shift,noise ...)
0.00

Nivel de ruido
0.00

Frecuencia de muestreo (sample rate)
44100

Ref. en decibelios para normalizar espectrogramas
-20

Normalización de audio (recomendable si los datos vienen de diferentes fuentes)
No

Parámetros de generación de mezclas

Coherent prob
0.50

[Guardar y volver](#) [Cancelar](#)

Figura 6.11: Parámetros de formación de transformadas de Fourier modificables en la interfaz visual.



Tamaño de la ventana STFT

512

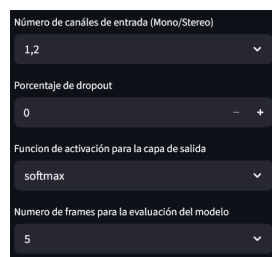
Desplazamiento entre ventanas STFT

256

Tipo de ventana utilizada para la STFT

hann

Figura 6.12: Parámetros de formación de las transformadas de Fourier modificables en la interfaz visual.



Número de canales de entrada (Mono/Stereo)

1,2

Porcentaje de dropout

0

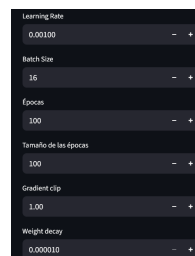
Funcion de activación para la capa de salida

softmax

Numero de frames para la evaluación del modelo

5

Figura 6.13: Parámetros intrínsecos del modelo modificables en la interfaz visual.



Learning Rate

0.00100

Batch Size

35

Epochs

100

Tamaño de las épocas

100

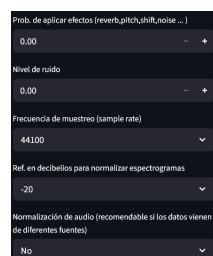
Gradient clip

1.00

Weight decay

0.000010

Figura 6.14: Parámetros modificables de entrenamiento de nuevos modelos en la interfaz visual.



Prob. de aplicar efectos (reverb, pitch, shift, noise ...)

0.00

Nivel de ruido

0.00

Frecuencia de muestreo (sample rate)

44100

Ref. en decibelios para normalizar espectrogramas

-20

Normalización de audio (recomendable si los datos vienen de diferentes fuentes)

No

Figura 6.15: Parámetros modificables de generación de efectos en la interfaz visual.



Figura 6.16: Parámetros modificables de generación de mezclas en la interfaz visual.

6.2.2. Casos de uso

El sistema permite entrenar modelos al persona usuaria utilizando el sistema definido en la anterior sección de este capítulo.

A partir de los parámetros definidos en configuración avanzada se hace llamada al mismo método que se ha utilizado para los entrenamientos de los modelos del proyecto y se comienza un nuevo ciclo de iteraciones de entrenamiento. Este nuevo modelo guardará sus *checkpoints* en una carpeta a parte de la de los modelos entrenados previamente para no pisar el trabajo realizado.

Esta funcionalidad permite al persona usuaria experimentar con el entrenamiento, ver cómo puede afectar la variación de hiperparámetros en los resultados y mostrar a alguien ajeno al tema qué es realmente un sistema de aprendizaje automático y como funciona.

Como el proceso de entrenamiento requiere de un gran tiempo de espera obtener un modelo entrenado, seguramente la persona usuaria no llegue a esperar lo suficiente para haber realizado un entrenamiento que ha aprendido lo suficiente como para generar resultados interesantes, se ha añadido esta funcionalidad porque se considera interesante que el persona usuaria pueda ver en tiempo real cómo es realmente un entrenamiento de un modelo de inteligencia artificial. El sistema permite el ajuste de una serie de parámetros para permitir al usuario jugar con estos y ver sus efectos en el modelo entrenado:

6.2.2.1. Parámetros de formación de las transformadas de Fourier

- Tamaño de la ventana STFT.
- Desplazamiento entre ventanas STFT.
- Tipo de ventana utilizada para la STFT.

6.2.2.2. Parámetros intrínsecos del modelo

- Número de canales de entrada (Mono / Estéreo).
- Porcentaje de *dropout*.
- Función de activación para la capa de salida.

6.2.2.3. Parámetros de efectos para entrenamiento y de audio

- Probabilidad de que se apliquen efectos a las entradas del modelo durante el entrenamiento (reverb,pitch,shift,noise ...).
- Nivel de ruido de fondo a añadir a las entradas del modelo durante el entrenamiento.
- Frecuencia de muestreo (*sample rate*).
- Referencia en decibelios para la normalización de espectrogramas.
- Normalización de audio (recomendable si los datos vienen de diferentes fuentes).

6.2.2.4. Parámetros de entrenamiento

- *Learning Rate*: controla cuánto se ajustan los pesos del modelo en cada iteración del entrenamiento.
- *Batch Size*: número de muestras procesadas antes de realizar una actualización a los pesos generados durante el entrenamiento.
- Épocas: número de veces que el algoritmo de entrenamiento recorre el conjunto de datos de entrenamiento.
- Tamaño de las épocas: número de muestras que se procesan en cada iteración.
- *Gradient clip*: límite a la magnitud de los gradientes para evitar actualizaciones excesivamente grandes e inestabilidad.
- *Weight decay*: penalización a los pesos demasiado grandes que reduce sobreajuste y favorece la generalización.

6.2.2.5. Parámetros de generación de mezclas

- *Coherent prob*: probabilidad de generar una mezcla coherente durante la generación de mezclas en el entrenamiento en la que todas las fuentes provengan de la misma canción y se alineen temporalmente.

6.2.2.6. Información sobre los parámetros en la interfaz

Además, se introduce otra pestaña adicional, mostrada en la Figura 6.17b y a la que se accede haciendo click en el botón 6.17a, donde se da información sobre lo que es cada parámetro



Figura 6.17: Entrada a la ventana de información sobre los parámetros de la configuración avanzada

6.2.3. Visualizaciones en la interfaz

El sistema muestra por pantalla una serie de visualizaciones una vez se separa una mezcla con el objetivo de proporcionar inspección visual de los resultados:

6.2.3.1. Waveforms

Una vez se separa una mezcla, se presenta por pantalla la forma de onda de cada una de las fuentes y de la mezcla. Ver Figura 6.18

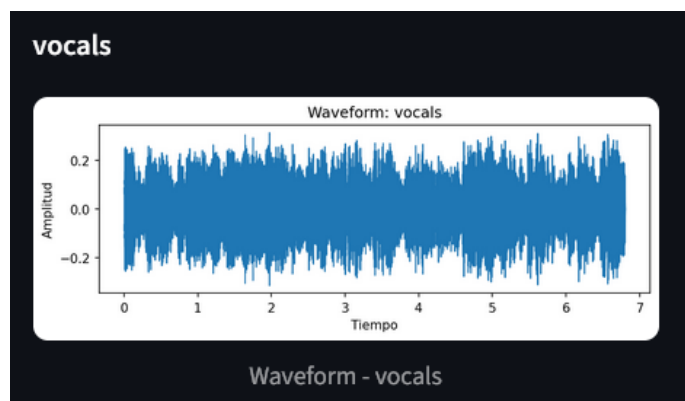


Figura 6.18: Ejemplo de una forma de onda mostrada en la interfaz al finalizar la separación de una pista.

6.2.3.2. Espectrogramas

A la vez que se muestra la forma de onda de la mezcla y de cada fuente, se muestra en conjunto el respectivo espectrograma. Ver figura 6.19

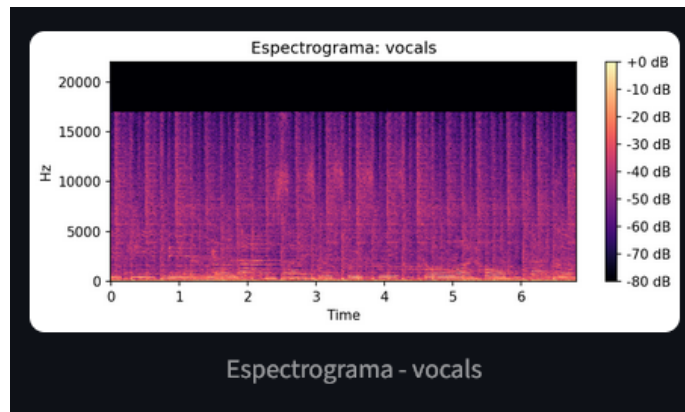


Figura 6.19: Ejemplo de un espectrograma mostrado en la interfaz al finalizar la separación de una pista.

6.2.4. Feedback

El sistema devuelve al usuario datos sobre las acciones que toma cuando estas generan salidas en la consola, se permite realizar una acción, se está realizando una acción o se necesita información que el usuario medio pueda desconocer.

6.2.4.1. Logs de consola

Cuando el sistema realiza un proceso, se imprimen por pantalla *logs* que puedan resultar informativos. Ver Figura 6.20

```
Consola y logs

[20:56:38] Iniciando separación de audio
[DEPLOY MODEL CONFIG]
hidden_size:
50
bidirectional:
True
num_layers:
1
[MODEL PATH]
```

Figura 6.20: Ejemplo de *logs* mostrados en la consola de la interfaz

6.2.4.2. Mensajes de error

El usuario puede realizar alguna acción que requiera algún paso previo. Si bien no se ha hecho un estudio exhaustivo de todas las situaciones que puedan desencadenar un error, el sistema muestra mensajes de error en la sección de la interfaz de la Figura 6.21 en caso de que se produzca alguno de los errores contemplados.

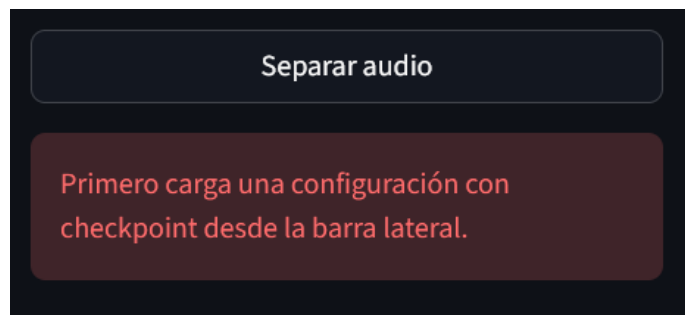


Figura 6.21: Ejemplo de mensaje de error mostrado en la interfaz al intentar separar una pista sin haber cargado una configuración de modelo previa.

Los errores que se han contemplado son los siguientes:

- No se encuentra ningún checkpoint para la configuración seleccionada.
- El checkpoint cargado ya no existe en el disco.
- Se intenta separar un audio sin haber cargado un checkpoint.
- Se intenta evaluar un modelo sin haber cargado un checkpoint.
- No existe alguno de los archivos de audio separados que se han generado.
- Falla la generación del waveform o espectrograma de una fuente.
- Incompatibilidad entre los pesos del checkpoint y la arquitectura del modelo.
- Claves faltantes, inesperadas o con dimensiones incompatibles al cargar un checkpoint.
- Fallo al escribir en disco.
- Formato o tipo de salida de audio en la separación no soportado.

6.2.4.3. Progreso

Mientras se está realizando una acción que requiere de tiempo, como separar una pista, la interfaz muestra un indicador de espera. Ver Figura 6.22.

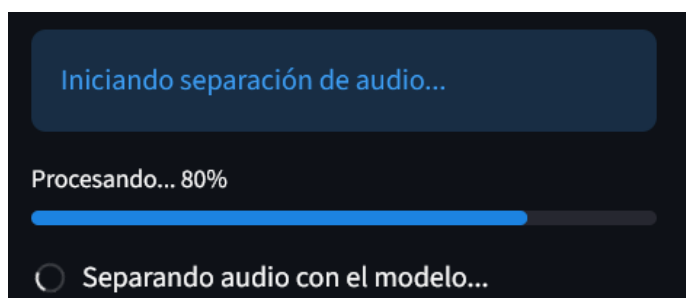


Figura 6.22: Ejemplo de indicador de espera en la interfaz mientras se está realizando el proceso de separar una pista.

6.2.4.4. Descarga de fuentes

Por cada fuente separada el sistema permite al usuario guardar una copia en su equipo. Esto estaría pensado para una situación real en la que este sistema está *hosteado* en un servidor y no en ejecución local ya que en ejecución local no hace falta descargar nada. Ver Figura 6.23.

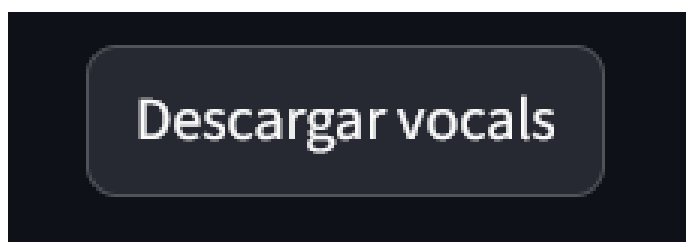


Figura 6.23: Botón de la interfaz que descarga un archivo wav de fuente en el almacenamiento del usuario.

El conjunto elegido e introducido en la sección anterior se separa en dos carpetas, *train* de 100 canciones y *test* con 50 canciones. La documentación indica que los enfoques supervisados deben ser entrenados en el conjunto de entrenamiento y probados en los conjuntos de test y de entrenamiento, y, además, que todas las pistas son estereofónicas y codificadas en 44.1kHz.

Para cada pista se tiene:

- *mixture.wav*: la mezcla original.
- *vocals.wav*: pista para las vocales.
- *drums.wav*: pista para la batería.
- *bass.wav*: pista para el bajo.
- *other.wav*: pista para elementos extra.

6. DESARROLLO

Los objetivos principales de la implementación para dar capacidades de interacción y aprendizaje se pueden resumir así:

- Cargar audio.
- Seleccionar modelo.
- Elegir función de pérdida/modelo entrenado. Proporcionar información de la elección.
- Seleccionar número de fuentes. Proporcionar información sobre la decisión.
- Separar fuentes.
- Reproducir resultados.
- Descargar stems.
- Visualizar waveform y espectrograma
- Opcionalmente entrenar un modelo propio con posibilidad de variar hiperparámetros. Proporcionar información sobre estos.

Integración del sistema

CAPÍTULO 7



En este capítulo se explica cómo se ha integrado el sistema , el backend y la comunicación entre los distintos módulos que lo componen.

7.0.1. Backend

El sistema está conectado internamente a partir de módulos. Estos son entrenamiento, inferencia, despliegue, evaluación e interfaz. La conexión principal es el archivo ***config.py***, el cuál contiene un diccionario de configuración que se va modificando a lo largo de las diferentes rutas de ejecución que parten desde *main.py*. El flujo de ejecución y comunicación entre módulos sigue el diagrama a continuación:

7. INTEGRACIÓN DEL SISTEMA

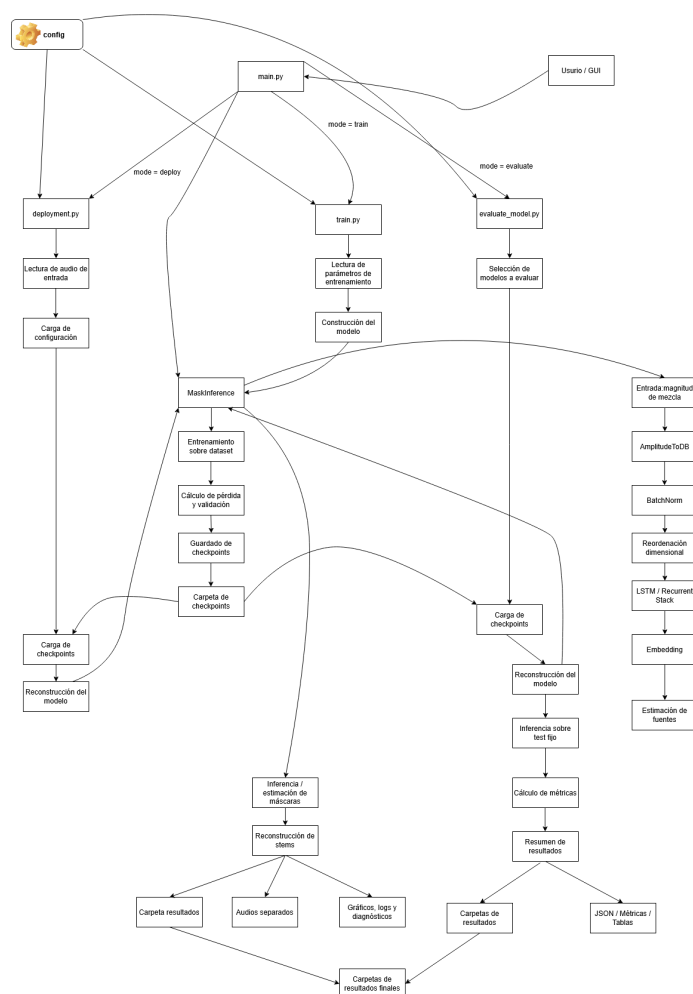


Figura 7.1: Diagrama de flujo para el backend

Y por módulos:

- **main.py** : punto de entrada principal, recibe por comando y ejecuta entrenamiento/despliegue/evaluación
- **config.py** : fuente central de parámetros, entre los que están parámetros de la STFT, hiperparámetros del modelo, función de pérdida, número de fuentes, rutas y dispositivo de ejecución.
- **train.py** : lee la configuración, construye un objeto **MaskInference**, entrena y calcula el valor de la pérdida y guarda checkpoints en la carpeta correspondiente.
- **deployment.py** : infiere la máscara inferida por el modelo entrenado sobre un audio concreto. Lee un audio de entrada, carga un checkpoint,

reconstruye el modelo, ejecuta inferencia, reconstruye las fuentes y guarda los resultados.

- `evaluate_models.py` : evalúa sistemáticamente los modelos que ya han sido entrenados sobre el conjunto de test fijo. Selecciona modelos, carga sus checkpoints, los reconstruye, ejecuta una separación sobre el conjunto de test, calcula las métricas de evaluación y guarda resúmenes en `.json`.
- `MaskInference` : contiene la arquitectura del modelo, la llamada a las transformaciones por cada iteración y la función de forward. Toma como entrada la magnitud de mezcla y aplica: `AmplitudeToDB`, `BatchNorm`, reordenación dimensional, `LSTM/RecurrentStack`, `Embedding`, máscaras estimadas y estimación de fuentes.
- Carpetas de checkpoints : guardan los archivos `.pth` ordenados por función de pérdida y por número de fuentes.
- Carpetas de resultados : guardan las salidas del backend. Contienen las fuentes separadas junto con la mezcla original, métricas, gráficos y ficheros resumen.

7.0.2. Comunicación entre módulos

La comunicación entre módulos es la definida en el diagrama de flujo que se presenta en la Figura 7.1.

El núcleo principal de la configuración entre módulos son los archivos `main.py` y `config.py`. En el archivo `main` se decide la acción que se va a ejecutar y se llama a una función que lleva a cabo la tarea requerida. Previamente se cambian valores del archivo `config`, como por ejemplo, la función de pérdida.

- 1. CLI / GUI modifica configuración
- 2. Se cargan datos o audio externo.
- 3. Se calcula STFT.
- 4. El modelo estima máscaras.
- 5. `run_inference` reconstruye audio.
- 6. `deployment` guarda stems.
- 7. Evaluación calcula métricas.
- 8. Resultados se guardan en carpeta de checkpoints/JSON/gráficas/tablas.

CAPÍTULO

Evaluación del sistema

8



En este capítulo se explica cómo se ha evaluado el sistema, las métricas utilizadas, la validación de resultados y los resultados obtenidos. Finalmente se analizan y se discuten estos resultados.

8.1. Métricas utilizadas

En esta sección se explicará qué se ha medido, por qué, y las limitaciones encontradas.

- **SI-SDR:** Como métrica principal se ha utilizado **SI-SDR** ya que es una métrica habitual en separación de fuentes y permite comparar la señal estimada con la señal de referencia. Mide cuánta parte de la estimación se alinea con la fuente objetivo, frente a la parte que puede considerarse distorsión.

A la hora de la interpretación de los resultados, para SI-SDR, se valora a mejor un valor mas alto de esta métrica mientras que un valor mas bajo implica peor separación.

$$\text{SI-SDR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{noise}}\|^2}$$

Donde:

- s_{target} es la señal de referencia.
 - e_{noise} es el error residual, es decir, todo lo que queda en la estimación y que no pertenece a s_{target} .
- **Baseline de mezcla:** Un modelo no se considera útil si no supera el la linea base de mezcla en el conjunto de test. Por ello, se ha añadido

un *baseline* trivial, donde se utiliza la mezcla original como estimación de cada fuente para así detectar modelos que generan audio, pero no separan de verdad, para ello se ha definido el un criterio mínimo para el valor de la correlación entre la mezcla y la fuente separada y eso es lo que determina el *baseline*. Se utiliza como criterio mínimo para calificar una fuente como separación y si no se supera quiere decir que el modelo genera audio pero no información útil.

■ Métricas de agregación:

Se han utilizado algunas métricas de agregación para tener información adicional sobre el rendimiento de los procesos realizamos, así como:

- Media: da una medida de rendimiento promedio para un conjunto de valores de, por ejemplo SI-SDR.
- Mediana: da un valor de robustez frente a *outliers*.
- Desviación típica: determina la distancia a la que están la mayoría de los valores de la media y se ha utilizado para valorar la estabilidad entre pistas.

■ Métricas complementarias:

Se han utilizado también varias métricas complementarias a modo de depuración y de diagnóstico: correlación entre fuentes de salida y mezcla, estadísticas de máscara y matriz de correlación estimación-referencia.

8.2. Validación del pipeline de evaluación

La validez de los resultados del enfoque investigador del proyecto se basa en la comparabilidad de estos mismos, y , por ello, la validación de estos debería de responder a las siguientes preguntas:

- ¿Con qué problemas nos encontrábamos antes en el proceso de evaluación?
- ¿Qué se hizo para validar la evaluación?
- ¿Por qué esto hace que los resultados sean comparables?
- ¿Qué se corrigió respecto a versiones anteriores?
- ¿Qué papel tiene el *baseline*?

Protocolo de evaluación utilizado:

El objetivo de crear un protocolo de evaluación no ha sido solo tener un proceso de ejecución sin errores, si no que también, y realmente de más importancia, asegurar que las métricas obtenidas reflejasen realmente las capacidades de cada función de pérdida en el modelo. El protocolo de evaluación propuesto es el siguiente:

- 1. División del *dataset* completo en el subgrupo *representative_test*, siendo este un conjunto fijo de test para cada modelo.
- 2. Evaluación de todos los modelos sobre las mismas pistas.
- 3. Normalización de referencias y estimaciones a mono-canal.
- 4. Se recortan las pistas a una longitud común
- 5. Se guarda un sumario en formato *JSON* para cada modelo.

Esto se ha hecho siguiendo las recomendaciones habituales de reproducibilidad en *Machine Learning* [24]: documentar dataset, *splits*, preprocesado, hiperparámetros y resultados. La checklist de reproducibilidad incluye precisamente detalles del dataset, particiones *train/validation/test*, datos excluidos y pasos de preprocesado.

Inicialmente el proceso aunque generase archivos no se daba la situación en la que los resultados fuesen realmente los buscados, en particular, porque las pistas de salida eran exactamente iguales a las pistas de entrada en la mayoría de los casos, por lo que algo se estaba haciendo mal. Para dar alguna medida de validez a los resultados obtenidos, se creó el subconjunto fijo de test desde el dataset, se creó la validación de *baseline*, se añadió una sección en el código donde se normalizaban las pistas en mono-canal, recorte de pistas a una longitud común, impresión de métricas de análisis de las máscaras por pantalla en el entrenamiento y normalización del flujo de inferencia para todas las funciones de pérdida, donde previamente existían variaciones.

A partir de aquí, todos los modelos pasan a evaluarse sobre las mismas pistas, con el mismo preprocesado, la misma métrica de evaluación y el mismo flujo de inferencia, además, al añadir la métrica *baseline* proporciona ayuda para identificar los resultados que generaban audios pero no separaban mas allá de copiar la mezcla de entrada.

El protocolo de evaluación final queda definido así:

1. Cargar el conjunto de test fijo *representative_test*.
2. Evaluar todos los modelos sobre las mismas pistas.
3. Ejecutar la inferencia mediante la misma función utilizada en el despliegue. Previamente se utilizaban funciones diferentes definidas en versiones anteriores.

8. EVALUACIÓN DEL SISTEMA

4. Convertir referencias y estimaciones a mono-canal.
5. Recortar referencias y estimaciones a una longitud común.
6. Mantener un orden fijo de fuentes para dos y cuatro *stems*.
7. Calcular SI-SDR para cada modelo y pista.
8. Comparar los resultados frente al baseline de mezcla.
9. Guardar los resultados y sumarios de evaluación en archivos estructurados.

8.3. Resultados obtenidos

Una vez definido y validado el pipeline de evaluación, se aplicó el protocolo anterior a los modelos entrenados. Los resultados se presentan separando los escenarios de dos y cuatro fuentes, ya que ambos problemas presentan distinta dificultad y no son directamente comparables.

En la Tabla 8.1 se presentan los resultados de separación en dos fuentes, mientras que en la Tabla 8.2 se presentan los resultados de separación en cuatro fuentes. En ambas tablas se muestran las métricas de SI-SDRi y SI-SAR, así como la desviación típica y si el modelo supera o no al *baseline*.

Tabla 8.1: Tabla para los resultados de separación en 2 fuentes

Función de pérdida	Grupo	Checkpoint	SI-SDRi medio	SI-SDRi mediano	Desviación típica	SI-SAR medio	Supera baseline
l1	principal	13	-7.157	-6.923	1.765	3.582	No
l2	principal	8	-2.515	-2.575	0.632	11.445	No
l1_freq	principal	13	-7.158	-6.924	1.762	3.578	No
l2_freq	principal	8	-2.262	-2.343	0.584	12.273	No
logl1	principal	69	-10.745	-11.134	2.771	-1.743	No
logl2	principal	77	-9.575	-9.118	2.675	0.012	No
log_mag	principal	56	-8.343	-8.071	2.066	2.130	No
log_compressed_l2	principal	45	-8.929	-8.540	2.198	0.500	No
lpsa	principal	61	—	—	—	—	—
mask_l1	principal	70	-8.239	-7.976	1.907	2.107	No
l_mrs	avanzado	—	—	—	—	—	—
lpsa_phase	avanzado	100	—	—	—	—	—
deep_feature	experimental	20	-0.091	-0.098	0.063	22.996	No
deep_feature_emd	experimental	21	-0.081	-0.082	0.042	23.435	No

Tabla 8.2: Tabla para los resultados de separación en 4 fuentes

Función de pérdida	Grupo	Checkpoint	SI-SDRi medio	SI-SDRi mediano	Desviación típica	SI-SAR medio	Supera baseline
l1	principal	24	-4.811	-5.085	1.650	-1.933	No
l2	principal	4	-0.065	-0.066	0.038	20.263	No
l1_freq	principal	24	-4.814	-5.089	1.651	-1.940	No
l2_freq	principal	4	-0.069	-0.070	0.037	20.437	No
logl1	principal	65	-9.356	-9.176	2.587	-8.803	No
logl2	principal	4	-5.231	-5.183	1.850	-2.760	No
log_mag	principal	61	-6.767	-6.589	2.151	-4.835	No
log_compressed_l2	principal	65	-6.801	-6.708	1.946	-5.555	No
lpsa	principal	—	—	—	—	—	—
mask_l1	principal	33	-6.424	-6.297	2.025	-4.469	No
l_mrs	avanzado	—	—	—	—	—	—
lpsa_phase	avanzado	99	—	—	—	—	—
deep_feature	experimental	1	-0.018	-0.020	0.055	16.532	No
deep_feature_emd	experimental	3	0.006	0.004	0.059	17.562	Sí

8.3.0.1. Rankings

En esta sección se presentan los rankings de los modelos entrenados, ordenados por la métrica de SI-SDRi medio, para los escenarios de dos y cuatro fuentes. Estos rankings permiten identificar rápidamente qué modelos han obtenido un mejor rendimiento en términos de separación de fuentes.

Tabla 8.3: Ranking para los resultados de separación en 2 fuentes

Rank	Función de pérdida	SI-SDR medio	SI-SDR mediano	SI-SDR STD	SI-SAR medio	Supera baseline
1	deep_feature_emd	-0.099	-0.109	0.063	23.435	No
2	deep_feature	-0.109	-0.104	0.091	22.996	No
3	l2_freq	-2.281	-2.349	0.563	12.273	No
4	l2	-2.533	-2.622	0.608	11.445	No
5	l1	-7.175	-6.975	1.737	3.582	No
6	l1_freq	-7.176	-6.976	1.734	3.578	No
7	lpsa_phase	-8.176	-8.721	1.915	—	No
8	mask_l1	-8.257	-8.027	1.871	2.107	No
9	log_mag	-8.361	-8.123	2.033	2.130	No
10	log_compressed_l2	-8.947	-8.556	2.164	0.500	No
11	lpsa	-9.423	-8.957	2.354	—	No
12	logl2	-9.593	-9.140	2.642	0.012	No
13	logl1	-10.763	-11.084	2.740	-1.743	No

8. EVALUACIÓN DEL SISTEMA

Tabla 8.4: Ranking para los resultados de separación en 4 fuentes

Rank	Función de pérdida	SI-SDR medio	SI-SDR mediano	SI-SDR STD	SI-SAR medio	Supera baseline
1	deep_feature_emd	-5.375	-5.260	0.440	17.562	Sí
2	deep_feature	-5.399	-5.320	0.413	16.532	No
3	l2	-5.445	-5.405	0.428	20.263	No
4	l2_freq	-5.449	-5.396	0.425	20.437	No
5	lpsa_phase	-6.423	-6.491	0.862	–	No
6	l1	-10.191	-10.449	1.727	-1.933	No
7	l1_freq	-10.194	-10.452	1.727	-1.940	No
8	logl2	-10.611	-10.651	1.880	-2.760	No
9	mask_l1	-11.805	-11.594	2.076	-4.469	No
10	log_mag	-12.147	-11.922	2.202	-4.835	No
11	log_compressed_l2	-12.181	-12.030	2.004	-5.555	No
12	logl1	-14.737	-14.475	2.562	-8.803	No

8.3.0.2. Graficos de barras

Se han generado dos gráficos de barras para dar una imagen que represente visualmente los resultados:

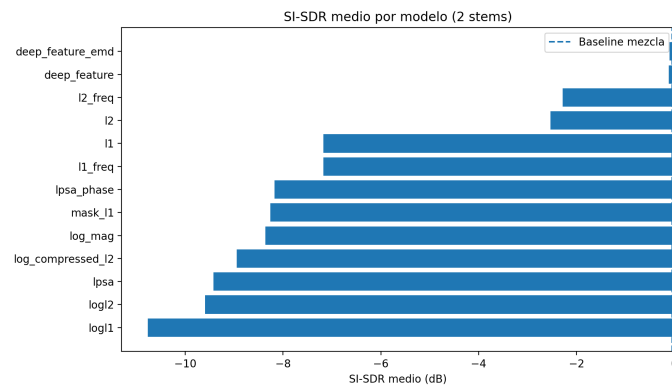


Figura 8.1: Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de dos fuentes

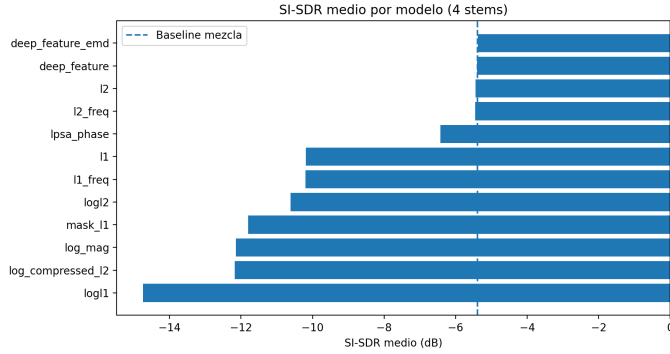


Figura 8.2: Gráfico de barras de los valores SI-SDR de cada función de pérdida en la separación de cuatro fuentes

8.3.1. Discusión y análisis

Los resultados obtenidos muestran diferencias claras entre las funciones de pérdida evaluadas, junto con un rendimiento limitado del sistema respecto al *baseline*. Para interpretar correctamente los valores proporcionados, es importante distinguir entre las métricas y el tipo de información que aporta cada una:

- **SI-SDR**: mide la calidad de la separación obtenida.
- **SI-SDR_i**: indica la mejora respecto a una referencia base.
- **SI-SAR**: permite observar la presencia de artefactos introducidos durante el proceso de separación, como ruido, distorsiones o presencia de señales de otras fuentes.

En el escenario de separación de dos fuentes, ninguna función supera el *baseline*. Los mejores resultados corresponden a:

- **Deep-Feature-EMD**: con un valor de SI-SDR_i medio de $-0,081$.
- **Deep-Feature**: con un valor de SI-SDR_i medio de $-0,091$.

Estos dos modelos son los más cercanos al *baseline*, aunque sin llegar a superarlo. Sus valores de SI-SDR_i están muy próximos a cero y, además, presentan los mejores valores de SI-SAR, con 23,435 y 22,996, respectivamente. Esto sugiere que son las configuraciones que introducen menos artefactos entre las evaluadas.

Por debajo de las funciones experimentales aparecen las pérdidas basadas en la norma L_2 , especialmente L_2 -Freq y L_2 . En este escenario, L_2 -Freq

obtiene un SI-SDRi medio de $-2,262$, ligeramente mejor que L2, que alcanza $-2,515$. Esto indica que la variante frecuencial mejora algo el resultado respecto a la versión básica en el escenario de dos fuentes.

Las funciones basadas en $L1$, las variantes logarítmicas y **Mask-11** obtienen los peores resultados. En particular, **log11** y **log12** presentan valores de SI-SDRi muy negativos, lo que indica un empeoramiento claro frente al *baseline*. Esto sugiere que las pérdidas logarítmicas no han proporcionado una señal de optimización adecuada para mejorar la separación.

En el escenario de separación de cuatro fuentes, los resultados son diferentes. Una función supera el *baseline*: **Deep-Feature-EMD**, con un SI-SDRi medio de $0,006$. Esta mejora es positiva, pero extremadamente reducida, por lo que debe interpretarse como una mejora marginal y no como una diferencia sólida. El valor obtenido por esta función está muy próximo a cero y, por tanto, no permite afirmar que el modelo supere de forma amplia al sistema de referencia.

Las funciones que ocupan las mejores posiciones en el ranking de SI-SDR medio son:

- **Deep-Feature-EMD**: con un SI-SDR medio de $-5,375$.
- **Deep-Feature**: con un SI-SDR medio de $-5,399$.
- **L2**: con un SI-SDR medio de $-5,445$.
- **L2-Freq**: con un SI-SDR medio de $-5,449$.

Resulta relevante observar el comportamiento del SI-SAR en cuatro fuentes. Las funciones **L2** y **L2-Freq** obtienen valores superiores a los de **Deep-Feature-EMD**, con $20,263$ y $20,437$, frente a $17,562$. Esto puede interpretarse como que, aunque **Deep-Feature-EMD** obtiene el mejor resultado en términos de mejora de SI-SDR, las pérdidas basadas en $L2$ generan separaciones con menor presencia relativa de artefactos.

Las pérdidas logarítmicas muestran un mal comportamiento en ambos escenarios. En cuatro fuentes, **log11**, **Log-mag** y **Log-compressed-12** se sitúan en las últimas posiciones del ranking y presentan valores negativos de SI-SAR, lo que indica una peor calidad de reconstrucción y una mayor introducción de artefactos. Este resultado sugiere que, para la arquitectura y el protocolo utilizados en este trabajo, estas pérdidas no se ajustan bien al objetivo de separación.

En conjunto, los resultados indican que las funciones experimentales basadas en características profundas son las más competitivas dentro del conjunto de modelos entrenados. En particular, **Deep-Feature-EMD** obtiene el mejor comportamiento global, al ser el único modelo que consigue superar, aunque de forma marginal, el *baseline* en cuatro fuentes y al quedar muy próximo a él en el escenario de dos fuentes.

La principal conclusión del estudio es que el proyecto ha permitido implementar y validar un pipeline completo de entrenamiento y evaluación para la separación musical, pero los resultados cuantitativos muestran que los modelos entrenados todavía presentan limitaciones. Las mejoras respecto al *baseline* son escasas, especialmente en el escenario de dos fuentes, y la única mejora positiva en cuatro fuentes es muy ligera.

Esto apunta a la necesidad de continuar ajustando la arquitectura, ampliar el entrenamiento, revisar las funciones de pérdida y estudiar estrategias adicionales para mejorar la calidad de separación. Los resultados obtenidos muestran diferencias claras entre las funciones de pérdida evaluadas, junto con un rendimiento limitado del sistema respecto al *baseline*. Para interpretar correctamente los valores proporcionados, es importante distinguir entre las métricas y el tipo de información que aporta cada una:

- **SI-SDR**: mide la calidad de la separación obtenida.
- **SI-SDRi**: indica la mejora respecto a una referencia base.
- **SI-SAR**: permite observar la presencia de artefactos introducidos durante el proceso de separación, como ruido, distorsiones o presencia de señales de otras fuentes.

En el escenario de separación de dos fuentes, ninguna función supera el *baseline*. Los mejores resultados corresponden a:

- **Deep-Feature-EMD**: con un valor de SI-SDRi medio de $-0,081$.
- **Deep-Feature**: con un valor de SI-SDRi medio de $-0,091$.

Estos dos modelos son los más cercanos al *baseline*, aunque sin llegar a superarlo. Sus valores de SI-SDRi están muy próximos a cero y, además, presentan los mejores valores de SI-SAR, con 23,435 y 22,996, respectivamente. Esto sugiere que son las configuraciones que introducen menos artefactos entre las evaluadas.

Por debajo de las funciones experimentales aparecen las pérdidas basadas en la norma $L2$, especialmente **L2-Freq** y **L2**. En este escenario, **L2-Freq** obtiene un SI-SDRi medio de $-2,262$, ligeramente mejor que **L2**, que alcanza $-2,515$. Esto indica que la variante frecuencial mejora algo el resultado respecto a la versión básica en el escenario de dos fuentes.

Las funciones basadas en $L1$, las variantes logarítmicas y **Mask-11** obtienen los peores resultados. En particular, **log11** y **log12** presentan valores de SI-SDRi muy negativos, lo que indica un empeoramiento claro frente al *baseline*. Esto sugiere que las pérdidas logarítmicas no han proporcionado una señal de optimización adecuada para mejorar la separación.

En el escenario de separación de cuatro fuentes, los resultados son diferentes. Una función supera el *baseline*: **Deep-Feature-EMD**, con un SI-SDRi

medio de 0,006. Esta mejora es positiva, pero extremadamente reducida, por lo que debe interpretarse como una mejora marginal y no como una diferencia sólida. El valor obtenido por esta función está muy próximo a cero y, por tanto, no permite afirmar que el modelo supere de forma amplia al sistema de referencia.

Las funciones que ocupan las mejores posiciones en el ranking de SI-SDR medio son:

- **Deep-Feature-EMD**: con un SI-SDR medio de $-5,375$.
- **Deep-Feature**: con un SI-SDR medio de $-5,399$.
- **L2**: con un SI-SDR medio de $-5,445$.
- **L2-Freq**: con un SI-SDR medio de $-5,449$.

Resulta relevante observar el comportamiento del SI-SAR en cuatro fuentes. Las funciones **L2** y **L2-Freq** obtienen valores superiores a los de **Deep-Feature-EMD**, con 20,263 y 20,437, frente a 17,562. Esto puede interpretarse como que, aunque **Deep-Feature-EMD** obtiene el mejor resultado en términos de mejora de SI-SDR, las pérdidas basadas en *L2* generan separaciones con menor presencia relativa de artefactos.

Las pérdidas logarítmicas muestran un mal comportamiento en ambos escenarios. En cuatro fuentes, **log11**, **log-mag** y **Log-Compressed-L2** se sitúan en las últimas posiciones del ranking y presentan valores negativos de SI-SAR, lo que indica una peor calidad de reconstrucción y una mayor introducción de artefactos. Este resultado sugiere que, para la arquitectura y el protocolo utilizados en este trabajo, estas pérdidas no se ajustan bien al objetivo de separación.

En conjunto, los resultados indican que las funciones experimentales basadas en características profundas son las más competitivas dentro del conjunto de modelos entrenados. En particular, **Deep-Feature-EMD** obtiene el mejor comportamiento global, al ser el único modelo que consigue superar, aunque de forma marginal, el *baseline* en cuatro fuentes y al quedar muy próximo a él en el escenario de dos fuentes.

La principal conclusión del estudio es que el proyecto ha permitido implementar y validar un pipeline completo de entrenamiento y evaluación para la separación musical, pero los resultados cuantitativos muestran que los modelos entrenados todavía presentan limitaciones. Las mejoras respecto al *baseline* son escasas, especialmente en el escenario de dos fuentes, y la única mejora positiva en cuatro fuentes es muy ligera.

Esto apunta a la necesidad de continuar ajustando la arquitectura, ampliar el entrenamiento, revisar las funciones de pérdida y estudiar estrategias adicionales para mejorar la calidad de separación.

8.3.2. Limitaciones y margen de mejora

A lo largo del desarrollo del trabajo, se ha tomado consciencia de las limitaciones que se pueden presentar en este tipo de tareas y que resultan en un peor desempeño del sistema.

- Fase de mezcla: como ya se ha comentado previamente, el uso de la fase de mezcla como estimación de la fase de las fuentes para reconstrucción limita en gran parte la tarea del modelo. Durante el entrenamiento y la inferencia, la red aprende a estimar máscaras aplicadas sobre la magnitud de la STFT de la mezcla, y posteriormente, se reutiliza la fase de la mezcla como aproximación para reconstruir la señal temporal de cada fuente.

Este procedimiento simplifica considerablemente el problema, ya que evita tener que modelar la componente de fase pero también introduce una limitación importante. La fase real de cada fuente, como ya se comenta previamente en la memoria, no tiene por qué coincidir con la fase de la mezcla por lo que esto puede resultar en la presencia de ruido o pérdida de calidad perceptual, especialmente en casos donde varias fuentes se solapan [5].

- Arquitectura simple: la arquitectura utilizada se basa en un modelo de inferencia de máscaras con una red recurrente LSTM. Esta selección permite trabajar con una estructura interpretable y adecuada para el procesamiento de espectrogramas como secuencias temporales y sin embargo se trata de una arquitectura sencilla en comparación con modelos actuales de separación musical, que suelen emplear redes más profundas, arquitecturas convolucionales o Transformers.
- Entrenamiento en mono: otra limitación relevante es que el sistema trabaja principalmente con señales mono-canal. Si bien esto reduce complejidad y facilita el entrenamiento, también implica la pérdida de información espacial de las señales estéreo. En música real, la presencia de instrumentos en los canales puede facilitar información útil a la hora de separar fuentes.

Si bien el procesamiento en mono-canal permite simplificar el problema y mantener un coste computacional asumible, también limita la capacidad del sistema para aprender sobre pistas especiales que podrían mejorar la calidad de la separación.

- Recursos computacionales: el entrenamiento de modelos de separación musical, y en general de modelos que trabajan con pistas de audio, requiere una cantidad considerable de recursos computacionales, especialmente cuando se trabaja con más de dos fuentes o funciones de pérdida complejas, como *l_mrs* o *lpsa_phase*.

Los recursos disponibles se limitaban a un procesador Intel de onceava generación y una tarjeta gráfica Nvidia RTX 3060Ti, que si bien no son recursos flojos, se quedan cortos para la tarea esperada. Algunos entrenamientos llegaron a tardar tres días y el tamaño del *batch* no era especialmente grande. Estas restricciones han condicionado decisiones como el tamaño del modelo, duración de fragmentos de audio, número de iteraciones de entrenamiento o cantidad de configuraciones evaluadas.

- Limitaciones del dataset: el rendimiento del sistema también lo condiciona el *dataset* utilizado. Si bien MUSDB18 es una referencia habitual, su tamaño es limitado en comparación con los utilizados para el entrenamiento de sistemas comerciales o modelos a gran escala.

Además, el dataset no cubre en su totalidad toda la variedad posible de géneros, mezclas, estilos de producción o calidades de grabación e instrumentación. Esto afecta a la capacidad de generalización del modelo principalmente. Si bien es complicado, ya que en la música, como en cualquier arte, no hay una norma fija que defina el objeto "música", existe una gran diversidad de estilos y enfoques y lo que para un artista en su proceso creativo es lo usual, para otro puede ser un completo disparate y ser igualmente válido en ambos casos y disfrutable para el oyente. La principal solución para este problema sería tener una cantidad de datos grande por género, por ejemplo, junto con otra cantidad grande de datos de géneros poco comerciales o experimentales. Esto permitiría que el modelo aprenda a generalizar mejor y no se vea limitado a un tipo de música concreta.

- Métricas objetivas y percepción auditiva: la evaluación del sistema se ha basado principalmente en métricas objetivas como SI-SDR que permiten comparar de forma cuantitativa las estimaciones generadas por los diferentes modelos. Se eligió utilizar estas métricas ya que se necesitaba una medida reproducible objetiva y homogénea para evaluar el rendimiento.

No obstante, las métricas objetivas no siempre reflejan completamente la percepción auditiva humana y de hecho, se repite en el estado del arte que las métricas numéricas no suelen ser una medida fiable de calidad y que es mejor conducirse por una opinión humana. Dos separaciones con valores similares de SI-SDR pueden percibirse de manera diferente en términos de ruido, naturalidad o inteligibilidad de la voz. Por ello, la escucha perceptual se consideró necesaria para interpretar los resultados. Se tenía intención de realizar una encuesta de evaluación pública con diferentes salidas del modelo para cada una de las funciones de pérdida, pero por falta de tiempo se terminó por dejar de lado.

- Interfaz demostrativa: la interfaz gráfica tiene un objetivo principalmente demostrativo. Su finalidad es dar facilidad para el uso del sistema a

un usuario ajeno al entorno de un IDE.

Aunque la interfaz permite la interacción directa con el pipeline y visualizar gráficos y logs, no debe entenderse como un producto comercial final, ya que no se han contemplado aspectos como control de errores, optimización de tiempos de respuesta, *hosting*, diseño visual de calidad o gestión de usuarios.

En conjunto, estas limitaciones no invalidan el sistema desarrollado, si no que más bien delimitan su alcance. El proyecto se centra en construir y validar un flujo completo de entrenamiento, inferencia y evaluación, sumándole a esto la interfaz.

Conclusiones y trabajo futuro

CAPÍTULO 9



9.0.1. Síntesis del trabajo realizado

9.0.2. Trabajo futuro

El sistema desarrollado ha cumplido con el objetivo de implementar un flujo completo de separación de fuentes musicales, incluyendo entrenamiento, inferencia, evaluación e interfaz gráfica. De todas formas, existen aun así diferentes líneas de mejora que permitirán ampliar el alcance del trabajo.

Se deja como trabajo futuro la incorporación de arquitecturas más avanzadas, datasets variados, entrenamiento estéreo y evaluaciones más valorables. Inicialmente, una primera fase de trabajo futuro sería realizar una comparación directa frente a otros modelos de mayor envergadura utilizados en separación musical para tener una referencia de alto nivel del desempeño de la función de pérdida que mejor desempeño presenta en este proyecto. En este trabajo dichos sistemas se han utilizado únicamente a modo de referencia histórica dentro del estado del arte, pero no se han tenido en cuenta para realización de una evaluación directa experimental bajo un mismo protocolo. Una comparación justa requeriría ejecutar todos los modelos sobre el mismo conjunto de test, con las mismas métricas y condiciones de evaluación, permitiendo así contextualizar de forma más precisa el rendimiento del sistema desarrollado.

Otra mejora considerable sería ampliar el conjunto de datos empleado. Si bien MUSDB18 presenta un conjunto de datos de referencia en separación de fuentes, su tamaño es limitado frente a colecciones más amplias utilizadas en, por ejemplo, competiciones como MDX.

Por otro lado, sería también un punto a favor estudiar la implementación de estructuras de arquitectura híbridas que combinen información en el dominio temporal con información en el dominio tiempo-frecuencia. El sistema trabaja con magnitudes de la STFT y aplica la fase de la mezcla sobre las magnitudes de las fuentes para reconstruir las fuentes. Los modelos híbridos aprovechan

la interpretabilidad de las representaciones espectrales y la capacidad de los modelos en forma de onda para aprender información temporal y de fase de forma más directa.

Otra mejora posible sería adaptar el sistema para la separación en tiempo real. La implementación actual está orientada a procesar archivos de audio completos o fragmentos previamente cargados. Para trabajar en tiempo real sería necesario rediseñar el flujo de inferencia, añadir procesamiento de audio por bloques, optimizar la carga del modelo y controlar solapamiento entre ventanas.

También se podría añadir al sistema la posibilidad de separar un mayor número de fuentes. En este trabajo se han reducido las posibilidades por limitaciones a dos y cuatro fuentes, pero existen pistas donde sería relevante separar instrumentos más específicos como guitarra, piano, sintetizadores, coros u otros elementos de la mezcla. Esta ampliación requeriría bases de datos con anotaciones de cinco fuentes, mayor capacidad de cómputo, modelos con mayor capacidad y una evaluación adaptada.

Finalmente, también se podría introducir la posibilidad de añadir efectos a las pistas de entrada para mejorar generalizaciones. La música contemporánea por lo general presenta una gran cantidad de post producción y cada caso es un mundo, se introducen muchos elementos que varían la producción inicial y esto varía en cada pista. Se planteaba inicialmente añadir esta posibilidad, pero por falta de tiempo y limitaciones no se pudo hacer.

Algunas de las posibilidades de efectos a aplicar que se plantearon eran:

- Reverberación
- Shiftteo de pitch
- Ecualización
- Ruido

Finalmente no se utilizó por problemas entre versiones de librerías. Se menciona aquí una visión positiva de esta complicación, pues así se permite mantener una comparación controlada entre funciones de pérdida.

10.0.1. Código relevante

El anexo de código recoge fragmentos representativos del sistema. No se incluirán archivos completos, sino secciones seleccionadas para dar luz sobre las partes más importantes del proceso

- **main.py:** punto de entrada principal del *backend*. Permite seleccionar modo de ejecución, y su relevancia reside en centralizar el enrutamiento hacia los diferentes módulos del proyecto.
- **MaskInference.forward():** se incluye para documentar el proceso de entrenamiento y validación. Contiene la construcción del modelo, carga de atos, cálculo del valor de la pérdida, control de entrenamiento, *early_stopping* y guardado de *checkpoints*.
- **metrics.py:** contiene las funciones de pérdida utilizadas. Es relevante para entender cómo se comparan las estimaciones del modelo con las referencias y cómo se implementan las variantes de pérdidas utilizadas.
- **commons/inference.py** proceso de inferencia completo.
- **commons/audio_utils.py:** se incluye por su papel en la carga de modelos e inferencia de parámetros. Resultó especialmente importante para resolver incompatibilidades entre configuración actual y modelos previos.
- **deployment.py:** módulo encargado de aplicar el sistema a audios. Permite cargar un archivo de entrada, ejecutar separación, reconstruir fuentes y guardar resultados.
- **evaluate_mixture_baseline.py:** identifica baseline utilizado.
- **GUI/gui.py:** contiene la implementación de la interfaz gráfica. Integra la implementación de la interfaz gráfica para selección de moelo, carga de

checkpoints, separación de audio, visualización de resultados, generación de gráficas, consola y descarga de resultados.

10.0.2. Configuraciones

A continuación se recogen ejemplos de comandos utilizados para ejecutar las funciones del sistema. Las rutas y nombres concretos se deben adaptar en función de la configuración local del proyecto.

10.0.2.0.1. Entrenamiento individual. Permite entrenar un modelo concreto a partir de la configuración seleccionada.

```
python main.py --mode train
```

10.0.2.0.2. Entrenamiento por lotes. Permite lanzar varios entrenamientos asociados a distintas funciones de pérdida o configuraciones.

```
python -m scripts.train_V2
```

10.0.2.0.3. Evaluación de modelos. Ejecuta la evaluación de los modelos entrenados sobre el conjunto de test fijo.

```
python -m scripts.evaluate_models
```

10.0.2.0.4. Evaluación del baseline de mezcla. Da el valor baseline trivial que se utiliza para determinar si un modelo entrenado mejora una solución mínima.

```
python -m scripts.evaluate_mixture_baseline
```

10.0.2.0.5. Separación sobre audio externo. Aplica el proceso a un archivo de audio de entrada.

```
python main.py --mode deploy
```

10.0.2.0.6. Ejecución de la interfaz gráfica. Lanza en local la interfaz.

```
streamlit run GUI/gui.py
```

10.0.2.0.7. Generación del conjunto de test representativo. Genera una copia local de un subconjunto representativo de MUSDB18.

```
python -m scripts.script_musdb
```

10.0.2.0.8. Diagnóstico de salidas separadas. Compara las fuentes estimadas con las referencias mediante una matriz de correlación.

```
python -m scripts.diagnose_track_outputs
```

10.0.2.0.9. Inspección de checkpoints. Permite cargar un *checkpoint* y mostrar la forma de algunas capas relevantes del modelo.

```
python -m scripts.test
```

10.0.2.0.10. Generación de medias, medianas y desviaciones. Permite obtener en tres archivos diferentes los valores de la media, mediana y desviación típica de la SI-SDR de cada modelo.

```
python -m scripts.sacar_medianas
```

10.0.3. Resultados adicionales

Bibliografía

- [1] Society of Broadcast Engineers. «The Physical Nature of Sound». En: *SBE Handbook Fundamentals of Audio* (2025). Accedido: 28 de agosto de 2025.
- [2] LibreTexts Physics. «Introduction to Sound». En: *LibreTexts Physics* (2025). Accedido: 28 de agosto de 2025.
- [3] OpenStax. «Sound Intensity and Sound Level». En: *OpenStax Physics* (2025). Accedido: 28 de agosto de 2025.
- [4] HuggingFace. «AudioCourse». En: (2023).
- [5] Z. Rafii et al. «An Overview of Lead and Accompaniment Separation in Music». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26.8 (ago. de 2018), págs. 1307-1335.
- [6] D.G. Childers; D.P. Skinner; R.C. Kemerait. *The cepstrum: A guide to processing*. 1977.
- [7] Monica Franzese y Antonella Iuliano. *Hidden Markov Models*. 2019.
- [8] Douglas Reynolds. *Gaussian Mixture Models*. 2009.
- [9] Juan Pablo Gascón. *ALGUNAS CONSIDERACIONES TEXTURALES EN LA REALIZACIÓN DE ARREGLOS PARA GUITARRA Y VOZ*. 2019.
- [10] DIEGO DE JESÚS SÁNCHEZ. *LA IMPORTANCIA DEL ACOMPAÑAMIENTO BATERÍSTICO EN GRAHAM: LA IMPROVISACIÓN COMO RECURSO CREATIVO*. 2021.
- [11] L. Rabiner; B. Juang. *An introduction to hidden Markov models*. 1986.
- [12] Fabian-Robert Stöter et al. «Open-Unmix: A Reference Implementation for Music Source Separation». En: *Journal of Open Source Software* 4.41 (2019), pág. 1667.

- [13] Romain Hennequin et al. «Spleeter: a fast and efficient music source separation tool with pre-trained models». En: *Journal of Open Source Software* 5.50 (2020), pág. 2154.
- [14] danielfrg. *Demucs - Music Source Separation*. 2020.
- [15] Alexandre Défossez et al. «Demucs: Deep Extractor for Music Sources with extra unlabeled data remixed». En: *arXiv preprint arXiv:1909.01174* (2019).
- [16] Zafar Rafii et al. *The MUSDB18 corpus for music separation*. Dic. de 2017.
- [17] Yashvi Chandola et al. *Deep Learning for Chest Radiographs: Computer-Aided Classification*. Academic Press, 2021.
- [18] Zafar Rafii et al. «An Overview of Lead and Accompaniment Separation in Music». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26.8 (2018), págs. 1307-1335.
- [19] Saarth Vardhan et al. «An Ensemble Approach to Music Source Separation: A Comparative Analysis of Conventional and Hierarchical Stem Separation». En: *Arxiv* ().
- [20] Bing Yang et al. «Supervised Direct-Path Relative Transfer Function Learning for Binaural Sound Source Localization». En: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2021.
- [21] Abhimanyu Sahai, Romann Weber y Brian McWilliams. «Spectrogram Feature Losses for Music Source Separation». En: *European Signal Processing Conference (EUSIPCO)*. 2019.
- [22] Shasha Xia, Hao Li y Xueliang Zhang. «Using Optimal Ratio Mask as Training Target for Supervised Speech Separation». En: (2017).
- [23] DeLiang Wang. «Time-Frequency Masking for Speech Separation and Its Potential for Hearing Aid Design». En: *Trends in Amplification* (2008).
- [24] Jpineau. «Machine Learning Paper Reproducibility Checklist». En: (2020).